



GRAVA: Grounded Reasoning-to-Action Representation and Learning for Autonomous Driving

Xiao Liu , Haoyu Li , Jianghao Leng , Lin Wang , *Member, IEEE*, Chao Sun , *Member, IEEE*

Abstract—Driving vision-language-action (VLA) models increasingly incorporate reasoning before action prediction, yet their intermediate reasoning is often weakly grounded in physical scene evidence and inconsistently connected to executable behavior. We present GRAVA, a framework centered on Grounded Reasoning-to-Action (GRA) that integrates grounding, reasoning, and action generation within a single autoregressive stream. GRA binds action-relevant linguistic references to 2D visual regions and ego-centric physical states, while organizing object-specific interactions and driving decisions in a trajectory-anchored typed graph. Its action-relevant structure is serialized into grounded reasoning, with bounding-box tokens and physical quantities providing evidence for interaction and decision statements. A single VLM generates this reasoning followed immediately by a compact Executable Planner action, which is deterministically decoded into a continuous trajectory. An agentic GRA data construction pipeline combines forward scene grounding with backward trajectory anchoring to derive consistent cognition and planning supervision from the same graph. Using this pipeline, we construct GR-NavSim, extending the nuPlan dataset with 2.2M grounded question-answer pairs and 70K GRA reasoning traces. Centered on the GRA representation, we propose a progressive training strategy that develops grounded cognition through pre-training, establishes the reasoning-to-action interface through lightweight imitation, and further improves driving behavior through reinforcement learning and exploration. Using only about 60% of the available human driving demonstrations for action supervision, GRAVA-8B achieves state-of-the-art performance among purely autoregressive driving models on the full NAVSIM benchmark. On an internal long-tail benchmark, full GRA improves key-object compliance and Closed-loop Driving Score by 19.3% and 20.5% relative to action-only prediction, respectively. These results demonstrate the benefit of preserving action-relevant physical evidence from grounded reasoning through executable action generation.

Index Terms—Vision-Language-Action Models, Autonomous Driving, Grounded Reasoning, Reasoning-to-Action, Motion Planning.

Xiao Liu, Jianghao Leng, and Chao Sun are with the National Engineering Research Center of Electric Vehicles, Beijing Institute of Technology, Beijing 100081, China; the Shenzhen Automotive Research Institute, Beijing Institute of Technology, Shenzhen 518118, China; and Shenzhen Jiguangzhijie Technology Co., Ltd., Shenzhen 518118, China (e-mail: bitliuxiao@gmail.com; lengjianghao@szari.ac.cn; chaosun@bit.edu.cn). Corresponding author: Chao Sun.

Haoyu Li is with the National Engineering Research Center of Electric Vehicles, Beijing Institute of Technology, Beijing 100081, China, and the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore (e-mail: rabbit.yujixyz@gmail.com).

Lin Wang is with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore 639798 (e-mail: linwang@ntu.edu.sg).

Code repository: <https://github.com/AhernResearch/grava>.

This work was supported by Shenzhen Science and Technology Program (ZDCY20250901103002003).

I. INTRODUCTION

VISION-LANGUAGE-ACTION (VLA) models are emerging as a promising paradigm for autonomous driving by integrating multimodal scene understanding, reasoning, and action generation within a general-purpose generative model. Recent driving VLAs further introduce explicit intermediate reasoning before action prediction to support decision making in complex and long-tail scenarios [1]–[3]. For driving, however, reasoning must identify not only the relevant traffic participants but also the physical states and interactions through which they constrain the ego behavior. A vehicle ahead, for example, affects the driving decision according to its lane occupancy, distance, and motion state. These scene-grounded relations must therefore remain explicitly connected to the action generated from the reasoning.

Despite this progress, existing driving VLAs still face two fundamental limitations, as illustrated in Fig. 1. First, we identify a **grounding gap (G1)** between linguistic reasoning and physical scene evidence. Action-relevant entities are often described through free-form language or high-level semantics without explicitly resolving them to visual regions and corresponding metric states. Consequently, a semantically plausible rationale may remain detached from the physical objects, distances, and motions that constrain the ego behavior. Second, we identify a **reasoning-to-action fragmentation gap (G2)**. Even when grounding is available, it is often introduced through dedicated perception or question-answering tasks, while reasoning and planning are learned from separate supervision or implemented through separate functional modules. The grounded objects and states established in one task are therefore not necessarily preserved as the evidence supporting subsequent interaction reasoning, decision making, and action generation. Addressing these limitations requires both explicit grounding of the reasoning content and continuity of that content through to executable behavior.

Establishing this continuity requires a representation that organizes scene evidence according to its role in the driving decision. Scene relevance is action dependent: visually salient objects are not necessarily those that constrain the demonstrated ego behavior, and different objects may impose distinct interaction requirements. Explicit visual references and physical states must therefore be connected to the object-specific interactions and decisions they support, rather than retained only as isolated perception attributes. This motivates an action-grounded representation that preserves the dependencies

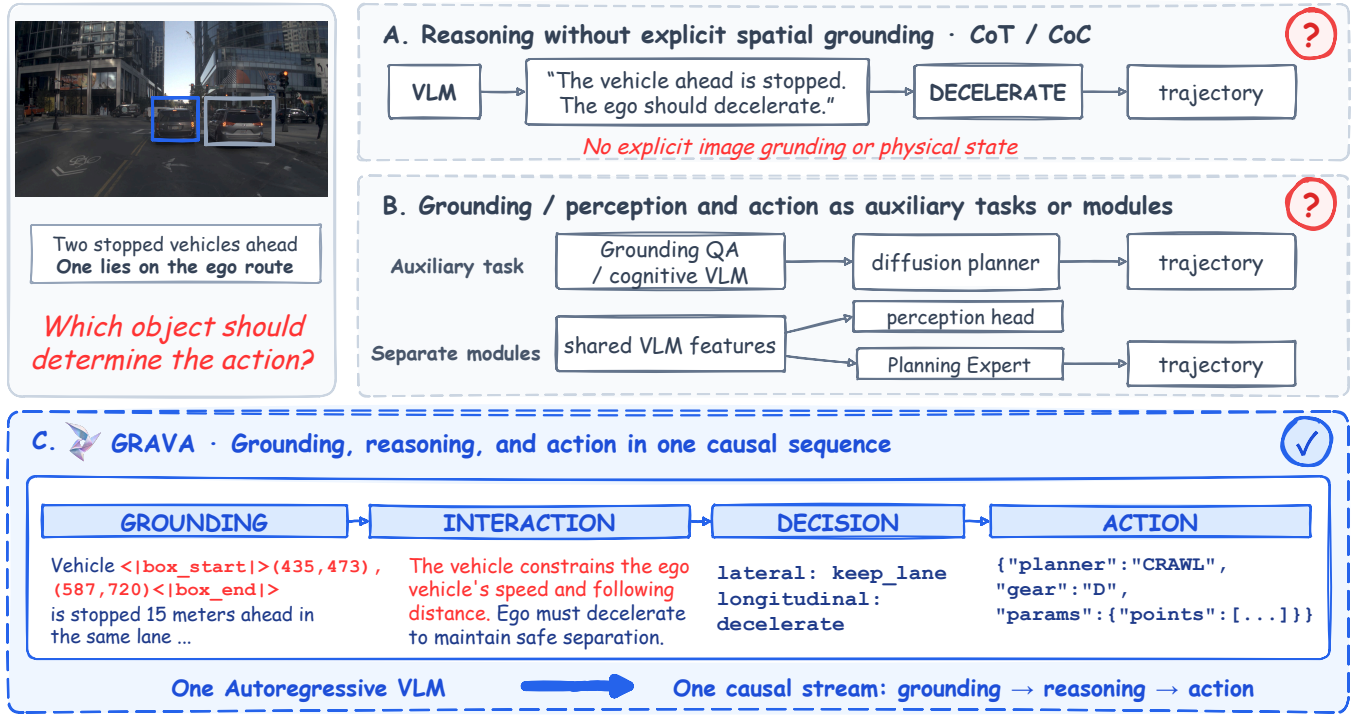


Fig. 1. Grounding and reasoning-to-action gaps in driving VLMs. (A) Reasoning without explicit visual grounding. (B) Grounding separated from action generation through auxiliary tasks or modules. (C) GRAVA connects grounded evidence, interaction reasoning, decisions, and executable actions within one autoregressive stream.

from physical scene evidence through reasoning to the final ego behavior.

To address this problem, we present **GRAVA**, a framework centered on the **Grounded Reasoning-to-Action (GRA)** representation. GRA organizes action-relevant scene context, grounded objects, physical states, interactions, decisions, and planning anchors within a trajectory-anchored typed graph. Each action-relevant linguistic referent is explicitly associated with its 2D visual region and ego-centric physical state, while object-specific interaction paths trace how this evidence contributes to the driving decision. The action-relevant graph structure is serialized into grounded reasoning in which bounding-box tokens and physical quantities remain associated with the interaction and decision statements they support. A single autoregressive VLM generates this reasoning followed immediately by a compact **Executable Planner** action, which is deterministically decoded into a continuous trajectory. GRA thus organizes the dependencies within the reasoning, while serialization and action generation carry them through the same autoregressive stream. Grounding becomes part of the reasoning used to generate the action, rather than an auxiliary task separated from downstream behavior.

Learning this grounded reasoning-to-action process requires supervision that preserves the same physical evidence and reasoning dependencies. GRAVA therefore employs a tool-augmented multi-agent annotation pipeline that combines forward scene grounding with backward trajectory anchoring. Forward grounding identifies candidate scene elements and establishes their visual and physical states, whereas backward anchoring starts from the expert trajectory and traces the

demonstrated behavior back to the critical objects, interactions, and decisions that support it. Cognition-oriented question answering and planning-oriented reasoning-to-action supervision are derived as different projections of the resulting GRA graph, maintaining consistent object identities, physical states, and interaction semantics across the two forms of supervision. Grounded concepts learned through individual questions thus reappear within the reasoning that supports executable action generation.

Based on this supervision, GRAVA adopts a GRA-aligned learning process that progressively connects grounded cognition to executable behavior. GRA pre-training establishes reference resolution, physical-state understanding, interaction reasoning, and decision semantics. Lightweight planner warm-up then learns to generate executable actions as continuations of grounded reasoning, and verified self-distillation further stabilizes valid model-generated reasoning. Finally, an **Active RL Loop** optimizes complete reasoning-to-action sequences using trajectory-level driving rewards. By repeatedly selecting recoverable scenarios in which high-reward behavior is already reachable but not yet reliable, Active RL concentrates policy refinement on unresolved scenarios while retaining grounded reasoning and action as a single optimization unit.

Our contributions are threefold:

- We propose **Grounded Reasoning-to-Action (GRA)**, an action-grounded representation that binds linguistic references to 2D visual evidence and ego-centric physical states, while organizing their interactions and decisions in a trajectory-anchored typed graph. Its action-relevant structure is serialized into grounded reasoning followed by

an Executable Planner action, preserving the progression from physical evidence to executable behavior within one autoregressive stream.

- We develop a **GRA-aligned pre-training and post-training strategy** that progressively learns and optimizes this reasoning-to-action process. Grounded pre-training, planner warm-up, and verified self-distillation establish the transition from scene understanding to executable action generation, while the Active RL Loop refines complete sequences using driving outcomes on policy-dependent recoverable scenarios.
- We develop an **agentic GRA data construction pipeline** and use it to build **GR-NavSim**, a vision-language alignment dataset based on nuPlan, comprising 2.2M grounded question-answer pairs and 70K GRA reasoning traces. To the best of our knowledge, GR-NavSim is the largest open-vocabulary, non-template-based question-answering dataset for autonomous driving to date.

We instantiate GRAVA-8B from Qwen3-VL-8B-Instruct [4]. Under the full NAVSIM protocol, GRAVA achieves 90.48 PDMS, improving by 10.2% relative to its pre-RL checkpoint and achieving the best performance among the directly autoregressive driving VLAs considered in our comparison. On an internal long-tail benchmark, full GRA improves key-object compliance and Closed-loop Driving Score by 19.3% and 20.5% relative to action-only prediction, respectively. Controlled reasoning interventions further show that replacing low-reward GRA reasoning with higher-quality grounded reasoning directly changes the generated action and substantially improves trajectory quality. These results support the central premise of GRAVA: action-relevant physical evidence should be explicitly grounded within the reasoning and preserved through interaction analysis, decision making, and executable action generation.

II. RELATED WORK

A. Grounded Reasoning in Driving VLAs

Grounded vision-language learning associates linguistic concepts with observable visual evidence. Visual Genome and GQA provide object-, relation-, and scene-graph-based supervision [5], [6], while MDETR and Grounding DINO explicitly connect language expressions to image regions [7], [8]. Region-aware multimodal models further support localized referring, question answering, and spatial reasoning [9]–[11]. These studies establish the visual reference resolution and spatial understanding capabilities needed to ground reasoning in scene evidence.

Driving VLMs and VLAs extend these capabilities toward reasoning about ego behavior. EMMA [1] jointly models scene understanding and trajectory generation, DriveLM [12] organizes driving cognition through graph-structured visual question answering, and DriveVLM [13], LMDrive [14], and Senna [15] introduce language-based reasoning or high-level decisions into planning. More recent methods explicitly incorporate physical grounding and interaction structure. OpenDriveVLA [2] uses 2D and 3D instance-aware representations with agent–environment–ego interactions, Alpamayo-R1 [16] constructs decision-grounded Chain-of-Causation

traces, ReCogDrive [17] develops hierarchical cognition before planning, and DriveAgent-R1 [18] invokes perception tools to ground decisions in visual evidence.

These approaches strengthen the connection between scene understanding and driving reasoning. GRA focuses on preserving action-relevant evidence throughout that reasoning: visual references and physical states remain associated with the object-specific interactions and decisions they support. Its trajectory-anchored graph organizes these dependencies, and its serialization retains them within the reasoning sequence that precedes executable action generation.

B. Reasoning-to-Action Representation

End-to-end autonomous driving has progressed from direct control and trajectory regression [19]–[21] toward object-centric, bird’s-eye-view, vectorized, and sparse representations that expose spatial structure to planning [22]–[27]. Diffusion and autoregressive planners further model multimodal trajectories and sequential motion [28]–[30]. For driving VLAs, the action interface must additionally connect language-based reasoning to continuous vehicle motion.

One approach retains a specialized trajectory generator alongside the VLM. DriveVLM [13] and Senna [15] use dedicated planning modules, ReCogDrive [17] couples cognition with a diffusion planner, and DriveVLA-W0 [31] introduces a lightweight action expert after world-model learning. Qwen-Drive-1.0 [32] integrates 3D perception and driving VQA within a shared VLM while generating trajectories through an external Planning Expert. These designs provide driving-specific motion generation through a learned module distinct from the autoregressive reasoning stream.

An alternative represents agent states, trajectories, or controls directly as token sequences [33]–[37]. AutoVLA [3] discretizes trajectories through an action codebook, while Curious-VLA [38] improves autoregressive trajectory generation through step-wise normalization and exploration. Related work also explicitly addresses language–action consistency: SimLingo [39] aligns visual question answering with closed-loop behavior, LinkVLA [40] links language and action through a shared discrete codebook, and Neuro-Symbolic Drive [41] derives rule-grounded reasoning traces from executable planners.

GRAVA follows the direct-action paradigm with a primitive-specific interface. The VLM generates an Executable Planner action immediately after grounded reasoning, and a fixed geometric decoder converts its parameters into continuous waypoints. The learned action choice therefore remains within $\pi_\theta(a \mid x, r)$, while geometric realization requires no additional learned trajectory policy.

C. Data Construction for Grounded Driving VLAs

Large-scale driving datasets such as BDD100K, nuScenes, Waymo Open, and nuPlan provide supervision for perception, prediction, and planning [42]–[45]. Language-oriented datasets extend this supervision toward command grounding and visual question answering [46]–[48]. DriveLM [12] organizes driving questions through graph dependencies, while NuPlanQA [49]

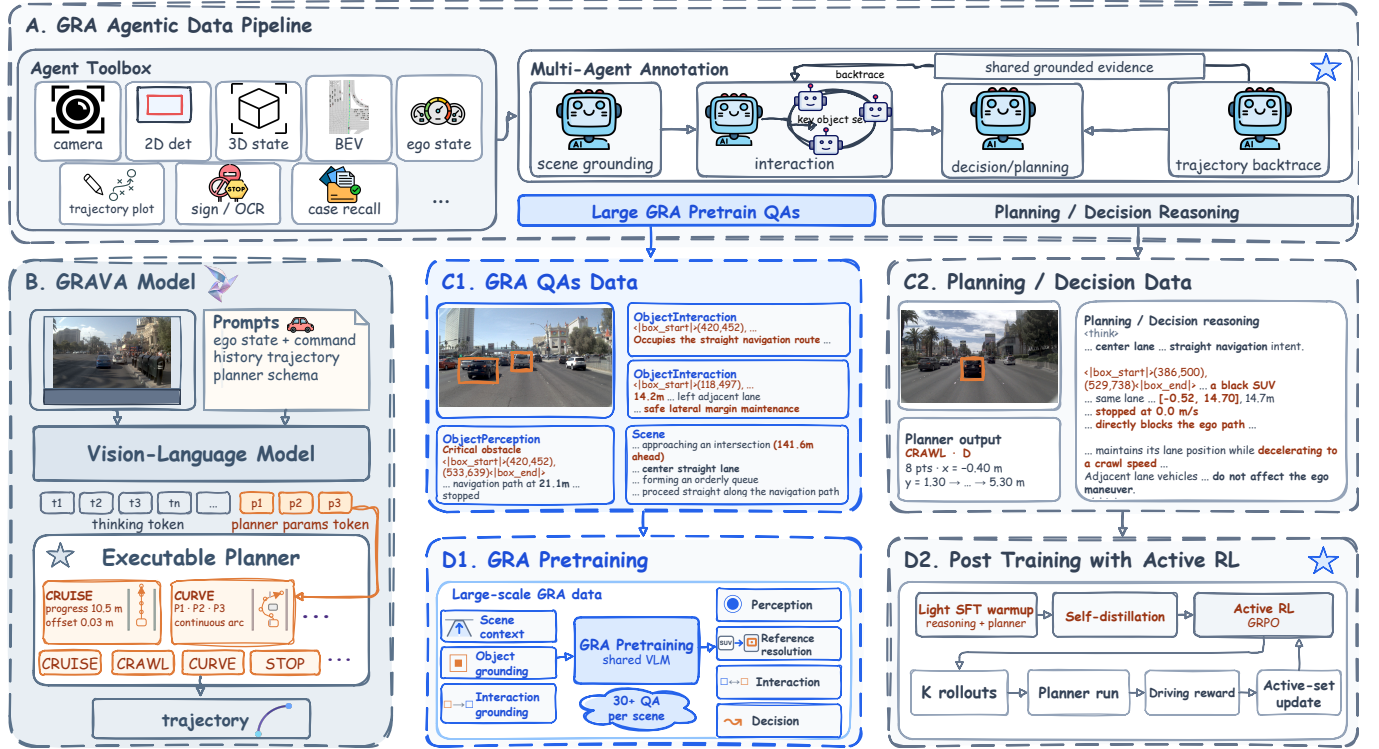


Fig. 2. Overview of GRAVA. (A) Agentic GRA graph construction through forward grounding and backward trajectory anchoring. (B) Autoregressive grounded reasoning and Executable Planner action generation, followed by fixed trajectory decoding. (C1–C2) Cognition and planning supervision derived from the same GRA graph. (D1–D2) Progressive training from grounded pre-training to Active RL.

provides multi-view scene-understanding and ego-centric reasoning supervision. NuScenes-QA further demonstrates how scene graphs constructed from existing 3D annotations can support scalable question-answer generation. Together, these datasets establish structured sources of driving cognition supervision.

Automated and model-assisted pipelines increasingly extend such supervision to driving reasoning. Alpamayo-R1 [16] constructs decision-grounded Chain-of-Causation traces through automatic and human-in-the-loop annotation, while ReCogDrive [17] uses a hierarchical cognition-data pipeline with generation, refinement, and quality control. These approaches improve the scale and organization of reasoning data. For learning a continuous reasoning-to-action process, a central consideration is how the grounded concepts acquired through cognition supervision remain consistent with the evidence used in planning targets.

The agentic GRA data construction pipeline addresses this cross-task consistency through a shared trajectory-anchored graph. Forward grounding establishes visual references and physical states, while backward trajectory anchoring identifies the objects, interactions, and decisions relevant to the demonstrated behavior. Cognition-oriented questions and planning-oriented reasoning sequences are derived from the same graph, preserving their object identities and decision semantics. The distinction lies in using the same grounded contents both as individual cognition targets and as the evidence supporting executable action learning.

Complementary evaluation work extends trajectory-displacement measures toward closed-loop and pseudo-closed-loop driving quality [50]–[53], providing behavioral criteria alongside language-oriented supervision.

D. Training Strategies for Driving VLAs

Training driving VLAs requires adapting general-purpose vision-language capabilities to driving behavior. Supervised fine-tuning learns scene understanding, reasoning, and action prediction from annotated examples [1], [2]. In imitation-centered training, the driving policy is primarily optimized to reproduce demonstrated behavior. This provides a direct route to acquiring driving skills, while its susceptibility to distribution shift motivates interactive imitation-learning methods such as DAGger [54].

A broader training strategy first establishes driving cognition and then adapts it to planning. ReCogDrive [17] adopts driving-specific pre-training followed by planner imitation and reinforcement learning. Qwen-Drive-1.0 [32] similarly combines perception and VQA adaptation with subsequent Planning Expert training and reward-based refinement. These staged systems distinguish the acquisition of scene-understanding capabilities from the optimization of driving behavior. Complementary self-training and reasoning-distillation methods reuse verified model-generated outputs to extend supervision beyond the original annotations [55], [56].

Within such training systems, RL and preference-based post-training support further policy improvement beyond supervised

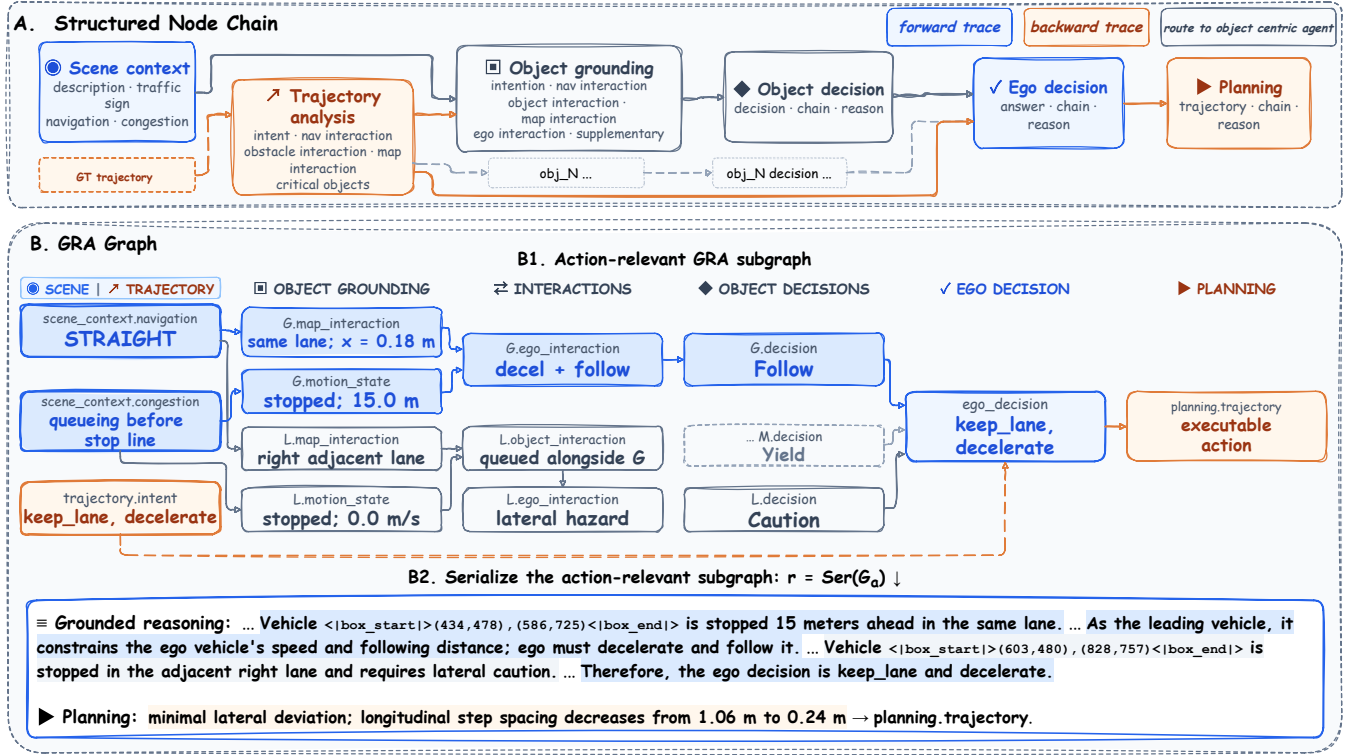


Fig. 3. GRA graph structure and serialization. (A) Semantic node types and their organization. (B1) Object-specific reasoning paths converge at the ego decision and connect to the action anchor. (B2) The action-relevant subgraph is serialized into grounded reasoning with visual references and physical states, followed by the planning target.

imitation [57]–[59]. Group-relative optimization, asymmetric policy updates, and entropy regulation provide mechanisms for learning from sampled outcomes [60]–[62]. In driving, AlphaDrive [63] applies group-relative optimization to reasoning, Gen-Drive [64] combines reward modeling with diffusion-policy fine-tuning, and AutoVLA [3] uses reinforcement fine-tuning for end-to-end VLA planning. Curious-VLA [38] further addresses policy narrowing after imitation through iterative exploration and sample refresh.

GRAVA emphasizes grounded cognition pre-training followed by lightweight planner imitation and reinforcement-driven exploration. GRA pre-training establishes the grounded concepts used in reasoning, while planner warm-up learns their continuation into executable actions. Verified self-distillation stabilizes model-generated reasoning before Active RL explores and optimizes complete $[r; a]$ sequences according to decoded trajectory outcomes. Imitation thus establishes the initial reasoning-to-action interface, while RL refines behavior on policy-dependent recoverable scenarios where high-reward actions are reachable but not yet reliable.

III. THE GRAVA FRAMEWORK

A. Overview and GRA Formulation

GRAVA generates a driving action by first identifying the relevant visual evidence and physical states, reasoning about how they constrain the ego vehicle, and then expressing the

resulting decision as an executable action. Grounding is part of this reasoning process: an object’s visual reference, distance, and motion state appear alongside the interactions and decisions they support. A single autoregressive VLM generates the grounded reasoning and the subsequent action, as illustrated in Fig. 2.

Grounded Reasoning-to-Action (GRA) defines how this information is organized. Its graph links scene context and grounded object evidence to interactions and driving decisions; its serialization expresses those dependencies as a reasoning sequence that the VLM can learn and generate. The graph thus specifies the structure of the reasoning, while the text is its sequential realization. We first describe this generation process, then explain how graph-derived supervision and staged learning establish it.

Given visual observation V_{env} , ego state s_{ego} , and navigation command c , the model input is $x = (V_{\text{env}}, s_{\text{ego}}, c)$. Let G_a denote the action-relevant GRA subgraph and $r = \text{Ser}(G_a)$ its grounded reasoning sequence. Bounding-box tokens identify relevant visual regions, physical attributes describe object states, and interaction and decision statements explain their influence on ego behavior.

After generating r , the VLM predicts an **Executable Planner** action $a = (p, g, \phi)$, where p is the motion primitive, g the gear, and ϕ the primitive-specific parameters. The complete sequence $y = [r; a]$ is generated autoregressively with the

factorization

$$\pi_\theta(y | x) = \pi_\theta(r | x) \pi_\theta(a | x, r). \quad (1)$$

The two factors describe a continuous information flow. The reasoning sequence introduces grounded objects and explains their behavioral relevance; the action is then generated with that reasoning in context. GRA addresses the grounding gap (G1) through the content and organization of r , and the reasoning-to-action fragmentation gap (G2) by learning a as its continuation within the same policy.

A fixed geometric decoder converts the action into a continuous trajectory:

$$\hat{\tau} = D_p(g, \phi; s_{\text{ego}}). \quad (2)$$

The decoder has no learnable parameters; the VLM selects the action, and D_p realizes its geometry.

Training supervision is derived from a target graph constructed offline:

$$G^* = \mathcal{A}(x, \tau^*, z), \quad (3)$$

where τ^* is the expert trajectory, z denotes privileged annotation evidence, and $\mathcal{A}$ is the agentic graph-construction process. Queries of G^* provide cognition-oriented QA targets, while its action-relevant paths provide reasoning targets paired with expert-derived planner actions. These targets teach the model both the individual grounded concepts and their use in a complete driving decision.

Section III-D describes how staged learning builds this capability and then improves it using driving outcomes. The graph-construction process is used offline; at inference, GRAVA receives only x and directly generates $[r; a]$. Neither an expert trajectory τ^* nor privileged evidence z is required to produce the reasoning and action.

B. Grounded Reasoning-to-Action Representation

GRA organizes scene evidence according to its role in the driving decision. In the example in Fig. 3, a vehicle is stopped 15m ahead in the ego lane. Its visual region identifies which vehicle is being discussed; its motion state and lane relation explain why it constrains forward motion; the resulting interaction calls for deceleration and following. A vehicle in the adjacent right lane imposes a different constraint, requiring lateral caution. GRA keeps these object-specific reasoning paths distinct before combining them into the ego decision.

The representation is *trajectory anchored*: the demonstrated behavior determines which scene evidence and relations are retained. We describe its node types and dependencies, explain how the graph is constructed, and then show how its contents become grounded reasoning and training supervision.

1) *Typed Grounded Reasoning Graph*: The nodes represent semantic contents such as navigation intent, object motion state, interaction, and decision. Figure 3 makes these contents explicit through fields such as `scene_context.navigation`, `G.motion_state`, and `G.ego_interaction`. Directed edges organize their dependencies from grounded evidence to object-level responses and the final ego decision. For each scene, this typed directed acyclic graph is defined as

$$G = (V, E, \nu, \eta), \quad (4)$$

where V denotes the node set, $E \subseteq V \times V$ the directed edge set, and ν and η specify the corresponding node and edge types. The node set is decomposed as

$$V = V^S \dot{\cup} V^O \dot{\cup} V^\Psi \dot{\cup} V^D \dot{\cup} \{v^\delta, v^a\}, \quad (5)$$

where V^S contains scene- and route-context nodes, V^O contains grounded object nodes, V^Ψ contains interaction nodes, and V^D contains object-level decision nodes. The ego-level decision node v^δ aggregates the responses induced by individual action-relevant objects into the final driving decision, while v^a serves as the terminal action anchor. The action anchor marks the endpoint of the grounded reasoning structure but does not itself contain the parameters of the Executable Planner.

The grounding information associated with an object connects its visual reference to the physical states used in subsequent reasoning. We collect this information in the object node o_i , writing, for each action-relevant object i ,

$$o_i = (m_i, b_i, \sigma_i), \quad (6)$$

where m_i denotes its linguistic reference, $b_i \in [0, 1000]^4$ is its normalized 2D bounding box, and σ_i denotes its ego-centric physical state. The state σ_i contains the semantic category and attributes such as relative position, distance, speed, and heading. The bounding box resolves which visible object a statement concerns; its physical state provides the information needed to reason about the object's effect on driving. Both remain associated with the object when it appears in later interaction and decision statements.

Let $\mathcal{C}$ denote the set of critical objects whose states or interactions constrain the demonstrated ego behavior. For each $i \in \mathcal{C}$, object-specific reasoning proceeds from scene context and grounded object evidence through one or more interaction nodes to an object-level decision:

$$(V^S, o_i) \longrightarrow \{\psi_i^\ell\}_\ell \longrightarrow d_i, \quad i \in \mathcal{C}. \quad (7)$$

The resulting object-level decisions are subsequently combined at the ego level:

$$(V^S, \{d_i\}_{i \in \mathcal{C}}) \longrightarrow v^\delta \longrightarrow v^a. \quad (8)$$

In the running example, the lead-vehicle branch contributes the need to decelerate and follow, while the adjacent-vehicle branch contributes lateral caution. Their combination supports the ego decision to keep the lane and decelerate. The branch-and-merge topology records which object supports each response before the responses are combined.

To prevent reasoning statements from becoming detached from observable evidence, GRA imposes structural grounding constraints. Every retained interaction or object-level decision must be traceable to at least one grounded object:

$$\forall v \in V^\Psi \cup V^D, \quad \text{Anc}_G(v) \cap V^O \neq \emptyset, \quad (9)$$

where $\text{Anc}_G(v)$ denotes the ancestor set of node v in G . Each object-level decision must also contribute to the ego-level decision:

$$\forall d_i \in V^D, \quad d_i \longrightarrow v^\delta, \quad (10)$$

and the ego-level decision must ultimately reach the action anchor:

$$v^\delta \longrightarrow v^a. \quad (11)$$

TABLE I
EXECUTABLE PLANNER ACTION SCHEMA AND PRIMITIVE-SPECIFIC
PARAMETER SPACES Φ_p IN THE EGO FRAME.

Primitive p	Parameters $\phi \in \Phi_p$	Shape	Motion regime
STOP	endpoint	$\mathbb{R}^2$	near-stationary or stop- ping motion
CRAWL	points	$\mathbb{R}^{8 \times 2}$	low-speed stop-and-go
CURVE	p_1, p_2, p_3	$\mathbb{R}^{3 \times 2}$	turning and high- curvature motion
CRUISE	progress_end, $v_{\text{end}}, \text{lat_end}$	$\mathbb{R}^3$	lane keeping and car following

where $\longrightarrow$ denotes directed path reachability. These constraints connect each retained interaction or object-level decision to grounded object evidence and link the object-level decisions to the final ego action anchor.

2) *Trajectory-Anchored Graph Construction*: The offline construction process in Eq. (3) combines two directions, illustrated in Fig. 2. *Forward scene grounding* establishes what is present: it identifies candidate traffic participants and scene elements and associates them with visual regions and physical states. This produces the evidence represented by V^S and V^O , but does not by itself determine which objects constrain the demonstrated action.

Backward trajectory anchoring establishes behavioral relevance: starting from the expert trajectory τ^* , it traces the demonstrated behavior back to the critical objects, interactions, and decisions that support it. In the running example, the lead vehicle’s stopped state and same-lane position provide the evidence for the deceleration-and-following response. Linking that response back to the grounded object makes its role in the demonstrated behavior explicit.

Together, the two directions connect observable scene evidence with action-relevant reasoning. Objects and relations that do not contribute to the demonstrated decision are excluded from the action-relevant structure. The resulting G^* records the grounded states, interactions, and decisions from which the learning targets are derived. Privileged evidence is used only in offline construction and training.

3) *Action-Relevant Graph Serialization*: Serialization makes the graph’s reasoning structure available to an autoregressive VLM. It expresses grounded states, interactions, and decisions in their dependency order, so that the model learns how physical evidence leads to a driving response. Let $\text{Anc}_G(v^a)$ denote the strict ancestor set of the action anchor. We extract the action-relevant reasoning subgraph as

$$G_a = G[\text{Anc}_G(v^a)], \quad (12)$$

where $G[\cdot]$ denotes the induced subgraph. The corresponding grounded reasoning sequence is obtained as

$$r = \text{Ser}(G_a). \quad (13)$$

Serialization follows the topological structure of G_a . Grounded objects are introduced with their linguistic references, normalized bounding-box tokens, and physical states. Subsequent statements retain the object references while describing

interactions and object-level decisions, before converging at the ego decision.

Figure 3(B2) illustrates this transition for the running example. The text first identifies the lead vehicle by its bounding box and states that it is stopped 15m ahead in the same lane. It then explains how that vehicle constrains speed and following distance, retains the adjacent vehicle’s lateral constraint, and concludes with lane keeping and deceleration. The graph nodes thus become connected reasoning statements, with the visual references and physical quantities embedded in the reasoning itself.

The action anchor itself is not included in the reasoning sequence r . Instead, as defined in Sec. III-A, the Executable Planner action a is generated immediately after r . The complete progression is therefore

$$G_a \longrightarrow r \longrightarrow a \longrightarrow \hat{\tau}. \quad (14)$$

This progression describes how the graph organizes the target reasoning and how that reasoning leads into action generation. At inference, the VLM directly generates the sequence: it introduces the grounded evidence, reasons about its consequences, and continues with the planner action. Section III-C details the action representation and its trajectory decoder.

4) *Shared Cognition and Planning Supervision*: The target graph supports two complementary forms of supervision. Cognition-oriented questions teach the model to identify and reason about individual parts of the structure; planning-oriented sequences teach it to use those contents together to produce an action. For cognition pre-training, graph queries sample a question q and answer u concerning grounded objects, physical states, interaction relations, or driving decisions. The corresponding objective is

$$\mathcal{L}_{\text{QA}} = -\mathbb{E}_{(x,q,u)} [\log \pi_\theta(u \mid x, q)]. \quad (15)$$

For planning-oriented learning, the action-relevant portion of the same target graph is serialized as

$$r^* = \text{Ser}(G_a^*), \quad (16)$$

and paired with the corresponding target Executable Planner action a^* . The planning objective is

$$\mathcal{L}_{\text{plan}} = -\mathbb{E}_{(x,r^*,a^*)} [\log \pi_\theta([r^*; a^*] \mid x)]. \quad (17)$$

Both objectives use the object identities, physical states, interactions, and decisions recorded in G^* . Grounding learned through a local question therefore appears again as part of the reasoning that supports an action. Shared supervision connects the acquisition of individual cognition capabilities to their use in the full reasoning-to-action sequence.

C. Executable Planner

Grounded reasoning explains how the scene constrains the ego behavior; the next step is to express that behavior as vehicle motion. The **Executable Planner** provides the action vocabulary for this continuation. The VLM selects a motion primitive and its parameters immediately after generating r , and a fixed geometric decoder converts this structured action into a trajectory. This keeps the transition from reasoning to action

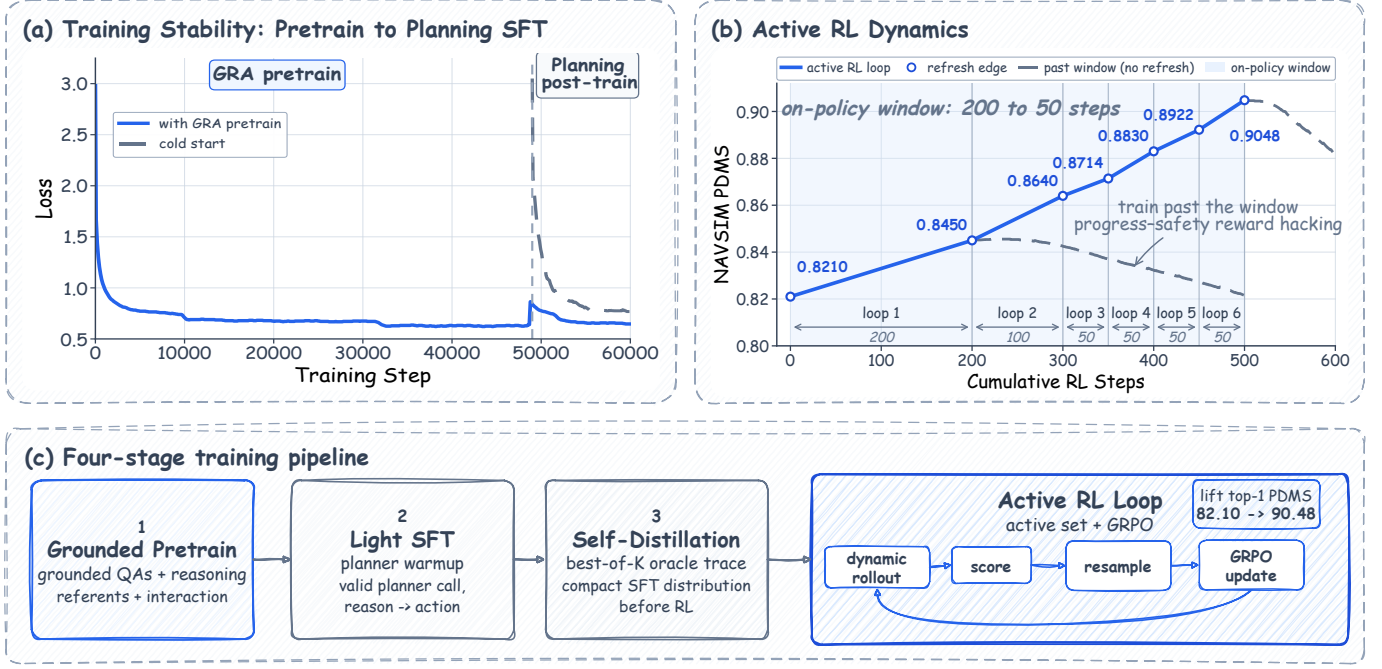


Fig. 4. Progressive training and Active RL dynamics of GRAVA. (a) Training loss with GRA pre-training and with planner training from a cold start (dashed). (b) Greedy NAVSIM PDMS across Active RL loops; shaded regions mark on-policy optimization windows. (c) Grounded pre-training, planner warm-up, verified self-distillation, and Active RL.

within the same autoregressive policy while allowing different motion regimes to use different geometric parameterizations.

The compact action is $a = (p, g, \phi)$. The primitive set $\mathcal{P}$ comprises STOP, CRAWL, CURVE, and CRUISE, with gear $g \in \mathcal{M} = \{D, R\}$ and parameters $\phi \in \Phi_p$. The executable action space is

$$\mathcal{A}_{\text{exec}} = \bigcup_{p \in \mathcal{P}} (\{p\} \times \mathcal{M} \times \Phi_p). \quad (18)$$

Table I summarizes the primitive-specific parameterizations. STOP represents near-stationary or stopping motion; CRAWL retains waypoint-level flexibility for irregular low-speed stop-and-go motion; CURVE describes turning and high-curvature trajectories; and CRUISE expresses lane-following and car-following motion through terminal motion variables.

The action is serialized as a continuation of the grounded reasoning:

$$\text{Ser}(a) = \{\text{planner} : p, \text{gear} : g, \text{params} : \phi\}. \quad (19)$$

The fixed decoder in Eq. (2) converts a valid action into an eight-waypoint trajectory $\hat{\tau} \in \mathbb{R}^{8 \times 2}$ over a 4-s horizon. Because the decoder has no learnable parameters, action selection remains in $\pi_\theta(a | x, r)$, with the grounded evidence and decisions in context. The Executable Planner is an action representation, not an additional learned planning policy.

1) *Supervised Action Target Construction*: During supervised learning, the target planner action $a^* = (p^*, g^*, \phi^*)$ is deterministically derived from the expert trajectory τ^* . The target encoder first assigns the motion primitive p^* according to the expert motion profile and navigation command, determines the driving gear g^* from the motion direction, and then fits only the parameters associated with the corresponding parameter

space Φ_{p^*} . The resulting parameters ϕ^* are quantized and serialized together with p^* and g^* to form the action tokens appended to the target grounded reasoning sequence r^* .

This target construction preserves the flexibility of each motion regime: CRAWL retains eight waypoints for irregular low-speed motion, while STOP, CURVE, and CRUISE use lower-dimensional parameters. Combined with the graph-derived reasoning, it yields the supervised sequence $y^* = [r^*; a^*]$, which teaches action prediction as a continuation of grounded reasoning.

D. Learning Grounded Reasoning-to-Action

GRAVA learns the reasoning-to-action process in stages, as illustrated in Fig. 4. GRA pre-training develops grounded cognition from graph-derived questions. Planner warm-up connects that cognition to executable actions through complete reasoning-to-action targets. Verified self-distillation then incorporates valid model-generated reasoning, and **Active RL** optimizes the resulting sequences using the driving outcomes of their decoded actions.

a) *Grounded pre-training and planner warm-up*: GRA pre-training uses the cognition objective in Eq. (15) to learn scene understanding, reference resolution, physical-state reasoning, interactions, and decision semantics. Planner warm-up then uses the sequence objective in Eq. (17) to learn how these contents support an executable action. Because both forms of supervision are derived from GRA, the grounded concepts learned through individual questions reappear in the reasoning that precedes the planner output.

Supervision establishes this mapping, but driving outcomes provide a further criterion for choosing among model-generated

sequences. Different actions can be geometrically plausible yet differ in safety, progress, and interaction behavior. The following stages first retain verified model-generated reasoning and then refine the complete $[r; a]$ sequence according to the quality of its decoded trajectory.

b) *Verified self-distillation*: Starting from the planner warm-up checkpoint, GRAVA samples complete reasoning-to-action sequences $y_i = [r_i; a_i]$. Candidates are selected according to the closed-loop driving rewards of their decoded trajectories. For each retained sample, the distilled target is defined as

$$y_i^+ = [r_i^+; a_i^+], \quad (20)$$

where r_i^+ and a_i^+ denote the reasoning and Executable Planner action from the same reward-selected rollout. The resulting checkpoint initializes Active RL and serves as its frozen reference policy.

c) *Active RL: selecting recoverable scenarios*: Active RL focuses on scenes where the current model can discover a good action but does not yet produce it reliably. Such *recoverable* scenarios provide contrasting reasoning-to-action sequences for the same observation: some lead to high-reward behavior, while others lead to substantially worse outcomes. For each input x , the current policy generates K complete rollouts $\{y_i = [r_i; a_i]\}_{i=1}^K$. Each valid planner action $a_i = (p_i, g_i, \phi_i)$ is deterministically decoded and evaluated using the driving reward

$$s_i = \mathcal{R}(D_{p_i}(g_i, \phi_i; s_{\text{ego}}); x). \quad (21)$$

We first identify scenarios that remain insufficiently solved under the policy's greedy output:

$$\mathcal{D}_g(\theta) = \{x : s_{\text{greedy}}(x) < \tau_g\}. \quad (22)$$

Within this candidate pool, a scenario is retained only when its sampled rollouts contain a sufficiently good solution, a substantially worse alternative, and meaningful reward variation:

$$\mathcal{D}_{\text{act}}(\theta) = \left\{ x \in \mathcal{D}_g(\theta) \left| \begin{array}{l} \max_i s_i \geq \tau_{\text{hi}}, \\ \min_i s_i < \tau_{\text{lo}}, \\ \text{std}_i(s_i) \geq \tau_{\sigma} \end{array} \right. \right\}. \quad (23)$$

The high-reward criterion selects scenes where improved behavior is reachable; the low-reward and variance criteria retain scenes where sampled outcomes still differ substantially. Together with the greedy filter, they identify where a successful sampled behavior has yet to become a reliable policy output. All candidate rollouts are generated by GRAVA itself, so this selection requires no additional action teacher.

d) *Outcome-based reasoning-to-action optimization*: For NAVSIM, $\mathcal{R}$ is defined as the PDMS of the trajectory decoded from each generated Executable Planner action. Outputs that cannot be parsed or violate the predefined executable schema receive zero reward. Active RL therefore evaluates generated actions according to their final driving outcomes rather than supervising motion primitives or planner parameters with manually designed intermediate rewards.

Because every rollout contains both grounded reasoning r_i and executable action a_i , the trajectory-level reward provides outcome feedback for the complete sequence $y_i = [r_i; a_i]$. Let $\bar{s}$ and σ_s denote the mean and standard deviation of the

rewards within a rollout group. The normalized group-relative advantage is

$$\hat{A}_i = \frac{s_i - \bar{s}}{\sigma_s + \epsilon}, \quad (24)$$

where $\epsilon > 0$ provides numerical stability. For token $y_{i,t}$, the importance ratio between the updated policy and the frozen behavior policy that generated the rollout is

$$\rho_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}. \quad (25)$$

Following DAPO-style asymmetric clipping [60], [61], the token-level objective is

$$\ell_{i,t}(\theta) = \min \left\{ \rho_{i,t}(\theta) \hat{A}_i, \right. \\ \left. \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon_-, 1 + \epsilon_+) \hat{A}_i \right\}. \quad (26)$$

The sequence-level Active RL objective is

$$\mathcal{J}_{\text{RL}}(\theta) = \mathbb{E}_{\substack{x \sim \mathcal{D}_{\text{act}}(\theta_{\text{old}}) \\ y_{1:K} \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)}} \left[\frac{1}{K} \sum_{i=1}^K \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \ell_{i,t}(\theta) \right. \\ \left. - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right]. \quad (27)$$

Here, π_{ref} is the frozen checkpoint obtained after verified self-distillation, β controls the KL regularization strength, and ϵ_- and ϵ_+ denote the asymmetric clipping bounds. The same rollout-level advantage $\hat{A}_i$ updates the token probabilities of both r_i and a_i . Driving feedback thus optimizes the grounded reasoning together with the action generated from it, retaining the complete sequence as the learning unit.

After each optimization window, the updated checkpoint is evaluated under greedy decoding and used to reconstruct the active set for the next loop. As the policy improves, scenarios that become reliably solvable are removed from the active set, while scenarios that remain recoverable but unstable determine the subsequent optimization distribution. The iterative process is summarized as

$$\pi_{\theta^{(k)}} \rightarrow \mathcal{D}_{\text{act}}^{(k)} \rightarrow \pi_{\theta^{(k+1)}}. \quad (28)$$

This alternation forms the Active RL Loop: trajectory rewards refine complete reasoning-to-action sequences, and renewed sampling identifies the next scenes on which to train. Across the learning stages, the model progresses from understanding individual grounded concepts to using them in action generation and refining that generation through driving outcomes.

IV. DATA AND EVALUATION PROTOCOLS

A. NAVSIM and GR-NavSim

NAVSIM provides real-world driving scenes and a held-out planning benchmark [51]. We construct **GR-NavSim** from its training scenes using the GRA-centered pipeline, obtaining 107K annotated scenes and 2.2M grounded question-answer pairs, together with Executable Planner examples and grounded trajectory traces. Cognition and planning targets share the same graph-derived object identities, physical states, interactions, and decisions. Table II summarizes the corpus, and Fig. 5 compares its scale with public driving QA datasets. Planning is evaluated

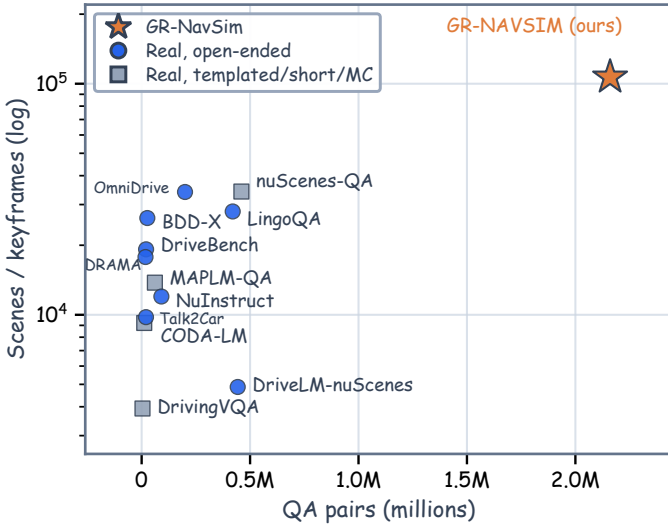


Fig. 5. Scale comparison of GR-NavSim and representative public driving QA datasets by QA count and source scenes or keyframes. The vertical axis is logarithmic; marker shapes distinguish answer formats.

on NAVSIM navtest under the official full protocol using the PDM Score (PDMS) and one greedy autoregressive completion per scene.

B. Internal Long-Tail Benchmark

Existing public driving benchmarks offer limited coverage of complex long-tail interactions. Models can achieve high scores primarily by fitting expert trajectories, making it difficult to fully assess the benefits of complex reasoning and reinforcement-driven exploration.

To evaluate these capabilities, we construct an internal long-tail benchmark covering route obstructions, lane borrowing, and road hazards. These scenarios require the ego vehicle to determine when to pass, when to wait, and how to plan a safe, feasible trajectory through the scene, accounting for obstacle states, available space, and interaction constraints. The benchmark contains 50K clips and 700K frames.

Each annotated key object is represented as $o_j = (t_j, \Omega_j, p_j)$, where t_j is its semantic type, Ω_j its manually annotated BEV region, and $p_j \in \{\text{passable}, \text{non-passable}\}$ its passability. This representation covers physical obstacles, construction regions, and road-surface hazards.

a) Closed-loop Driving Score: The logged future trajectory is not necessarily the only valid solution for a scene, as it may reflect conservative waiting behavior or characteristics of the original data-collection policy. We therefore roll out each predicted trajectory and evaluate its safety, Key-Object Compliance (KOC), route progress, and comfort. Safety is a hard gate, $\text{Safety} = G_{\text{col}}(\tau)G_{\text{nds}}(\tau)$: the Collision Gate equals one for rollouts without at-fault collisions, and the Non-Driveable-Space Gate equals one when the ego footprint remains within drivable space. The aggregate score is

$$\text{CDS} = \text{Safety} (0.45 \text{ KOC} + 0.35 \text{ Progress} + 0.20 \text{ Comfort}). \quad (29)$$

Progress is the maximum route displacement of the ego front edge, normalized by the maximum reachable distance

TABLE II
COMPOSITION OF THE GR-NAV SIM TRAINING CORPUS.

Corpus component	Count	Role
Driving scenes with grounded state labels	107K	Grounded objects, physical states, interactions, and decisions
Grounded question-answer pairs	2.2M	Grounded scene understanding and interaction reasoning
Front-camera Executable Planner examples	415K	Planner warm-up and self-distillation
Grounded trajectory traces	70K	GRA paths paired with planner actions

and clipped to $[0, 1]$; Comfort measures dynamic feasibility and smoothness. The larger KOC weight emphasizes correct responses to interaction-critical objects.

b) Key-Object Compliance: A pass-or-hold annotation specifies the desired interaction decision but does not by itself determine whether a generated trajectory actually executes that decision. KOC therefore converts each annotated key-object label into a geometric constraint on the simulated trajectory. This formulation allows multiple geometrically different trajectories to satisfy the same interaction constraint rather than requiring agreement with a single logged future trajectory.

We project each key-object region onto the route as $I_j = [s_j^-, s_j^+]$. Only objects intersecting the route interaction corridor within the evaluation horizon are retained; unreachable pass-required objects are excluded. The applicable pass and hold constraints are

$$\begin{aligned} c_j^{\text{pass}} &= \mathbb{I}[s_{\text{ego,front}}^{\text{max}} > s_j^- + d_{\text{trigger}}], \\ U &= \max(s_j^- - d_{\text{follow}}, U_{\text{min}}), \\ c_j^{\text{hold}} &= \mathbb{I}[s_{\text{ego,front}}^{\text{max}} \leq U], \end{aligned} \quad (30)$$

where $s_{\text{ego,front}}^{\text{max}}$ is the maximum route progress of the ego front edge and j_0 is the nearest hold-required object. The margins d_{trigger} , d_{follow} , and U_{min} are fixed across models. Only pass-required objects before the nearest hold boundary contribute, allowing a trajectory to pass an earlier object and stop behind a later one. For the applicable constraint set $\mathcal{K}$,

$$\text{KOC} = \frac{1}{|\mathcal{K}|} \sum_{c \in \mathcal{K}} c. \quad (31)$$

Progress is set to zero if the ego vehicle crosses a hold boundary. Samples without applicable key-object constraints are excluded from KOC aggregation, and all models use the same relevance thresholds and invalid-sample filtering rules.

V. EXPERIMENTS

We evaluate the central premise of GRA: physical evidence should remain connected to interaction reasoning and executable behavior. The experiments examine overall driving performance, the contributions of the representation and staged learning, and how grounded cognition supports the generated actions.

TABLE III
SINGLE-SAMPLE PERFORMANCE COMPARISON ON NAVSIM NAVTEST V1, GROUPED BY MODEL DESIGN.

Method	Backbone	Views	NC	DAC	EP	TTC	Comf	PDMS ↑
<i>Non-VLA methods</i>								
UniAD [25]	—	Cam	97.8	91.9	78.8	92.9	100.0	83.4
PARA-Drive [65]	—	Cam	97.9	92.4	79.3	93.0	99.8	84.0
TransFuser [66]	—	C+L	97.7	92.8	79.2	92.8	100.0	84.0
DiffusionDrive [28]	—	C+L	98.2	96.2	82.2	94.7	100.0	88.1
<i>VLA with additional trajectory head or world model</i>								
DriveVLA-W0 [31]	Emu-3-8B	Cam	98.7	99.1	87.6	97.1	100.0	90.2
ExploreVLA [67]	—	Cam	98.8	98.4	83.5	96.5	99.9	90.4
ReCogDrive [17]	InternVL2-8B	Cam	98.2	97.8	83.5	95.2	100.0	89.6
<i>VLA with direct action generation</i>								
AutoVLA one-shot [3]	Qwen2.5-VL-3B	Cam	96.9	92.4	75.8	88.1	99.9	80.5
AutoVLA Post-RFT [3]	Qwen2.5-VL-3B	Cam	98.4	95.6	81.9	98.0	99.9	89.1
Curious-VLA [38]	Qwen2.5-VL-3B	Cam	98.4	96.9	88.5	97.9	98.1	90.3
AdaThinkDrive [68]	InternVL3-8B	Cam	98.4	97.8	84.4	95.2	100.0	90.3
GRAVA before Active RL	Qwen3-VL-8B	Cam	96.1	91.2	78.8	91.0	99.7	82.1
GRAVA after Active RL	Qwen3-VL-8B	Cam	98.8	97.6	83.5	97.1	100.0	90.5

GRAVA is evaluated before and after Active RL under the same inference protocol. Cam and C+L denote camera-only and camera-plus-LiDAR inputs. NC: no-at-fault collision; DAC: drivable-area compliance; EP: ego progress; TTC: time to collision; Comf: comfort; PDMS: PDM Score. Higher is better for all metrics.

TABLE IV
COMPONENT ABLATION ON NAVSIM NAVTEST UNDER THE FULL SINGLE-SAMPLE EVALUATION PROTOCOL.

Pre-training	Reasoning condition	Action representation	SD	Active RL	NC	DAC	EP	TTC	Comf	PDMS ↑
None	Full GRA	Executable Planner	✓	✓	95.3	90.0	75.4	90.9	99.8	79.80
GRA	Full GRA	Direct waypoints	✓	✓	97.7	95.0	82.4	94.2	99.9	87.23
GRA	Full GRA	Executable Planner	—	—	96.0	91.3	77.3	91.7	99.9	81.74
GRA	Full GRA	Executable Planner	✓	—	96.1	91.2	78.8	91.0	99.7	82.10
GRA	Full GRA	Executable Planner	✓	✓	98.8	97.6	83.5	97.1	100.0	90.48

Each scene uses one greedy completion. SD denotes verified self-distillation. Metric abbreviations follow Table III; higher is better for all metrics.

TABLE V
ABLATION OF PRE-TRAINING, REASONING STRUCTURE, AND ACTIVE RL ON THE INTERNAL LONG-TAIL BENCHMARK.

Pre-training	Reasoning condition	Active RL	Col.	NDS	KOC	Progress	Comfort	CDS ↑
None	Full GRA	✓	95.8	94.7	81.7	89.6	98.8	79.7
Ungrounded	Full GRA	✓	96.3	95.4	84.2	90.7	99.1	82.2
GRA	Action only	✓	94.6	93.2	77.4	86.9	97.9	74.8
GRA	Coarse reasoning	✓	95.2	94.1	80.6	88.4	98.6	77.9
GRA	Grounded objects only	✓	97.1	96.4	87.9	91.3	99.3	85.5
GRA	Full GRA	—	93.1	94.9	69.8	90.1	98.9	73.1
GRA	Full GRA	✓	98.2	97.6	92.3	92.9	99.6	90.1

All variants use the Executable Planner and verified self-distillation. Col. and NDS denote the pass rates of the Collision Gate and Non-Drivable-Space Gate. KOC: Key-Object Compliance; CDS: Closed-loop Driving Score. Higher is better for all metrics.

A. Setup

GRAVA uses Qwen3-VL-8B-Instruct [4] as its backbone and follows the GRA-aligned training procedure in Sec. III-D. Only about 60% of the available human driving demonstrations are used for action supervision. Unless otherwise specified, all main results use a single greedy autoregressive completion per scene; multiple samples are used for self-distillation, diagnostic analyses, and Active RL. The generated Executable Planner action is converted to a trajectory by the fixed decoder. Benchmark definitions and evaluation protocols follow Sec. IV.

B. Main NAVSIM Results

We first assess the overall driving performance of the GRA framework. With a single camera and one greedy completion

per scene, GRAVA achieves 90.48 PDMS on NAVSIM navtest. It obtains the highest PDMS among the directly autoregressive VLAs in Table III and remains competitive with systems that introduce an additional trajectory head or world model. The table groups methods by model design to place this result in context, with differences in backbone, sensor inputs, and training data.

This performance is achieved by generating grounded reasoning and executable actions within one autoregressive policy, with no learned trajectory head or world model. Active RL further improves this policy using the driving outcomes of its generated sequences, while the visual input, greedy decoding procedure, and fixed trajectory decoder remain unchanged. We next examine how the representation and its training stages contribute to this performance.

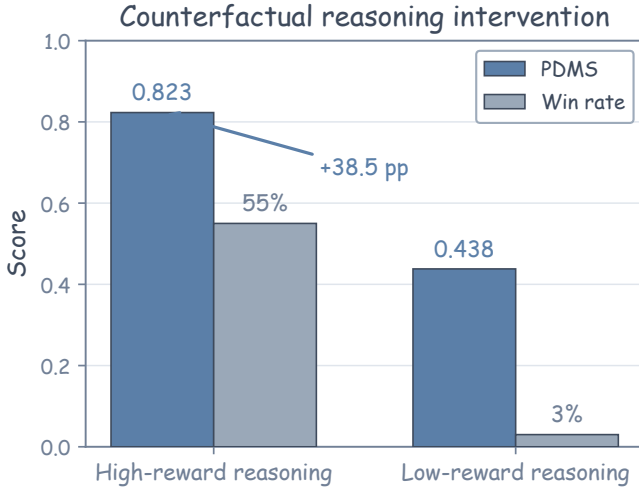


Fig. 6. Effect of reasoning intervention on action generation. Normalized PDMS and win rate are compared under high- and low-reward GRA reasoning, with the scene input and deterministic trajectory decoder held fixed.

C. Component Ablation on NAVSIM

Table IV relates the overall performance to two aspects of the framework: learning grounded cognition and its action interface, and refining the resulting policy through post-training. All variants follow the same NAVSIM evaluation protocol.

GRA pre-training provides an important foundation for planning. Removing it reduces PDMS by 10.68 points, from 90.48 to 79.80. Figure 4(a) also shows lower loss at the start of planning training with GRA pre-training than when training the planner from a cold start. These observations support the role of grounded cognition before learning to use it for action generation. Replacing the Executable Planner with direct waypoint generation reduces PDMS by 3.25 points, indicating that the primitive-specific action representation helps translate the learned reasoning into vehicle motion.

Verified self-distillation provides a further 0.36-point improvement before RL. The resulting policy can already generate high-reward behaviors in some scenes, but does not yet produce them reliably. Active RL improves this policy from 82.10 to 90.48 PDMS by optimizing complete reasoning-to-action sequences using driving outcomes. Figure 4(b) shows gains across successive active loops, while continued training beyond an on-policy window without refreshing the active set leads to declining performance. This behavior supports refreshing the recoverable scenarios as the policy changes, as described in Sec. III-D. The internal benchmark further examines how this learning process affects decisions in complex interactions.

D. Internal Long-Tail Results

The internal benchmark evaluates decision-making and the ability to navigate complex long-tail scenes involving route obstructions, lane borrowing, and road hazards. The ego vehicle must decide when to proceed or wait and plan a safe, feasible trajectory that carries out that decision. Following Sec. IV-B, Key-Object Compliance (KOC) measures the execution of

TABLE VI
GROUNDED QUESTION-ANSWERING PERFORMANCE ON THE HELD-OUT GR-NAVSIM EVALUATION SET.

Subtype	Qwen3-VL-8B	Kimi-K2.5	GPT-5.4	GRAVA
<i>Scene</i>				
Summary	6.09	6.03	7.08	7.47
Traffic sign	4.74	5.58	5.87	7.63
Navigation	6.21	6.44	6.33	6.67
Congestion	5.41	6.79	6.59	7.92
Average	5.61	6.21	6.47	7.42
<i>Object perception</i>				
Critical object description	2.01	2.19	2.79	5.71
Critical object identification	1.32	2.80	3.71	5.69
Average	1.65	2.51	3.27	5.70
<i>Object interaction</i>				
Intention	4.69	5.04	6.37	6.13
Navigation interaction	4.20	4.60	6.17	6.49
Object interaction	4.00	3.31	5.06	5.23
Map interaction	4.68	4.65	5.62	6.73
Ego interaction	4.27	4.03	4.94	6.00
Average	4.37	4.32	5.63	6.12
<i>Decision (label accuracy)</i>				
Object decision	3.16	3.68	4.21	8.34
Ego decision	6.05	6.32	5.13	8.66
Overall	4.40	4.77	5.39	6.86

Scene, object-perception, and object-interaction subtypes use 0–10 scores; decision subtypes use label accuracy. Average and Overall denote category and aggregate scores, respectively. Qwen3-VL-8B is the untuned backbone. Bold indicates the best result; higher is better.

interaction decisions, while CDS evaluates overall driving performance. We vary the pre-training target, reasoning structure, and use of Active RL while keeping inference fixed, with results reported in Table V.

Grounded pre-training provides stronger benefits than ungrounded cognition supervision. Ungrounded pre-training improves CDS by 2.5 points over no pre-training, while GRA pre-training further improves CDS by 7.9 points and KOC by 8.1 points. This comparison extends the NAVSIM ablation by showing the value of grounding the cognition learned before planning, particularly for responding to interaction-critical objects.

The reasoning structure provides benefits beyond object grounding alone. Full GRA improves KOC and CDS by 14.9 and 15.3 points over action-only prediction, and by 4.4 and 4.6 points over the grounded-objects-only variant. The latter comparison directly addresses the role of the interaction and decision paths in Sec. III-B: identifying an object supplies the evidence, while reasoning about its effect on the ego route supports decisions about whether and how to proceed. The gains in both metrics indicate better execution of these decisions and improved driving performance through the scene.

Active RL further improves the policy’s ability to carry out these decisions in complex long-tail scenes. KOC increases from 69.8 to 92.3 and CDS from 73.1 to 90.1, whereas Progress and Comfort improve by 2.8 and 0.7 points. These gains reflect better execution of appropriate pass-or-hold behavior and improved overall driving performance. The larger increase in KOC indicates that the benefit mainly lies in carrying out

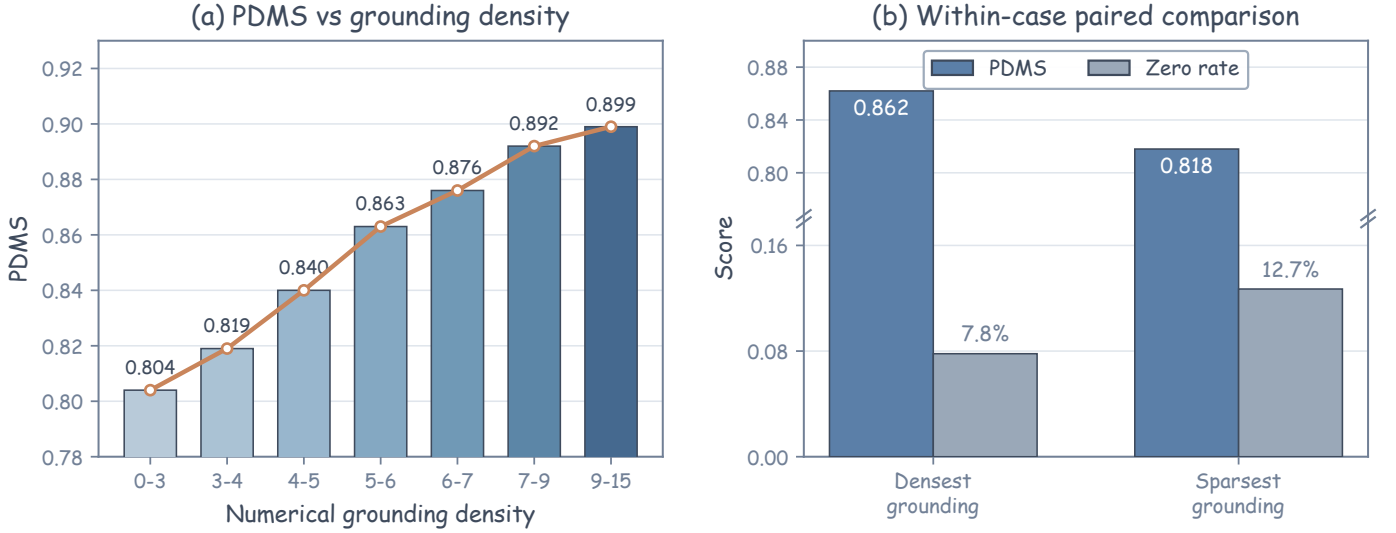


Fig. 7. Numerical grounding density and planning quality. Density counts references to action-relevant object distances and speeds. (a) Mean normalized PDMS across density bins. (b) Within-scene comparison of the densest and sparsest completions in normalized PDMS and zero-PDMS rate.



Fig. 8. Qualitative GRA examples on NAVSIM (top) and the internal long-tail benchmark (bottom). Each example links grounded scene evidence to interaction reasoning, driving decisions, and an Executable Planner action.

the appropriate maneuver, rather than simply advancing farther through the scene.

E. Grounded Reasoning Analysis

The long-tail results motivate a closer examination of how GRA reasoning uses physical evidence and influences the action generated from it. We also evaluate the grounded cognition capabilities that support this process.

a) *Effect of Reasoning on Action Generation*: GRA learns action generation as a continuation of grounded reasoning

(Sec. III-A). To examine whether this preceding reasoning affects behavior, we hold the scene input and deterministic trajectory decoder fixed and replace only the reasoning sequence. As shown in Fig. 6, replacing low-reward reasoning with its high-reward counterpart changes the predicted action and increases normalized PDMS from 0.438 to 0.823, while the win rate rises from 3% to 55%. This intervention provides direct evidence that the generated action depends on the reasoning in its context, supporting the reasoning-to-action connection proposed to address **G2**.

b) *Numerical Grounding*: Having established that reasoning affects action generation, we examine the physical information used within it. Numerical grounding density counts references to the distances and speeds of action-relevant objects, corresponding to the physical-state evidence introduced to address **G1**. Figure 7(a) shows that normalized PDMS increases from 0.804 in the lowest-density bin to 0.899 in the highest. The within-scene comparison in Fig. 7(b) follows the same trend: the densest completion averages 0.862 normalized PDMS with a 7.8% zero-score rate, compared with 0.818 and 12.7% for the sparsest. Both comparisons associate explicit physical-state grounding with better planning outcomes. The measured content concerns object distances and speeds, rather than the overall length of the reasoning trace.

c) *Grounded Question-Answering Performance*: The preceding analyses examine reasoning during planning; the held-out GR-NavSim QA set evaluates the individual cognition capabilities underlying that reasoning. This follows the shared-supervision design in Sec. III-B, where questions teach grounded concepts that reappear in the complete reasoning-to-action sequence. We compare GRAVA with GPT-5.4 [69], Kimi-K2.5 [70], and the untuned Qwen3-VL-8B-Instruct backbone. Open-ended responses are scored by an independent LLM judge, while decision questions are evaluated using label accuracy. All models receive the same ego-frame coordinate prompt for localization.

GRAVA achieves the highest overall score of 6.86 and leads on 12 of the 13 subtypes (Table VI); GPT-5.4 performs slightly better only on intention prediction. The strongest gains over the untuned backbone concern critical-object perception and object-level decisions. For example, the critical-object identification score rises from 1.32 to 5.69, while the object-decision score rises from 3.16 to 8.34. These improvements concern recognizing the relevant physical evidence and determining the response required by each object. Together with the planning ablations, they support learning these grounded concepts through shared supervision and using them in subsequent action generation.

F. Executable Planner Analysis

Grounded cognition must ultimately be expressed as vehicle motion. The Executable Planner ablation establishes the benefit of this action interface; we now examine how its parameters account for differences in trajectory quality. Among low-score cases with a large reward gap between sampled actions, high- and low-reward trajectories use the same motion primitive in most cases. Their differences lie in endpoints, lateral offsets, speed profiles, and CRAWL waypoints. Selecting the motion regime therefore leaves an important part of the driving decision to be resolved through its continuous parameters.

Within the same motion primitive, driving rewards distinguish parameter choices that lead to different trajectory quality. The primitive-specific action space in Sec. III-C therefore allows Active RL to refine how a behavior is executed through its endpoints, lateral offsets, or speed parameters. This complements the improved compliance with interaction constraints observed on the internal benchmark.

G. Qualitative Grounded Reasoning-to-Action Analysis

Figure 8 brings these analyses together through complete GRA examples. In the NAVSIM queue scene, grounding identifies a stopped vehicle in the ego lane and a crossing rider. The interaction reasoning assigns different constraints to these objects: the vehicle blocks forward progress, while the rider introduces a crossing conflict. Their combined effect supports keeping the lane and decelerating to a controlled crawl, which is expressed by the CRAWL action. The same object evidence is thus preserved from reference resolution through interaction reasoning to executable behavior.

The remaining cases illustrate different consequences of this structure. An open-door roadside hazard requires lateral clearance and produces a CURVE trajectory. On the internal benchmark, a red signal and stopped lead vehicle require a STOP action, whereas a moving lead vehicle and roadside construction call for following with lateral clearance, expressed through CRUISE parameters. Across these cases, GRA connects what the model observes to why a response is needed and how it is executed.

VI. CONCLUSION AND FUTURE WORK

We presented GRAVA, a Grounded Reasoning-to-Action framework that connects physical scene evidence, structured reasoning, and executable behavior through a shared GRA representation. GRA binds action-relevant linguistic references to visual regions and physical states, while organizing interactions, decisions, and planning anchors in a trajectory-anchored graph. The same representation supports both cognition and planning supervision, while GRA-aligned pre-training, the Executable Planner, verified self-distillation, and Active RL progressively transform grounded reasoning into executable actions. The resulting GRA-centered data pipeline further produces GR-NavSim with 107K grounded scenes and 2.2M grounded question-answer pairs. GRAVA-8B achieves 90.48 PDMS on NAVSIM, while full GRA improves Key-Object Compliance and Closed-loop Driving Score by 19.3% and 20.5% relative to action-only prediction, respectively. These results demonstrate the benefit of constructing grounding, reasoning, and action learning around a shared action-relevant representation.

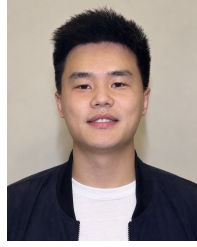
Future work will extend GRAVA toward temporal and multi-view grounding, interactive closed-loop environments, longer-horizon behaviors, and more efficient reasoning through distillation and adaptive computation.

REFERENCES

- [1] J.-J. Hwang *et al.*, “Emma: End-to-end multimodal model for autonomous driving,” *Transactions on Machine Learning Research*, 2025.
- [2] X. Zhou, X. Han, F. Yang, Y. Ma, V. Tresp, and A. Knoll, “Opendrivevla: Towards end-to-end autonomous driving with large vision language action model,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 16, 2026, pp. 13 782–13 790.
- [3] Z. Zhou *et al.*, “Autovla: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [4] S. Bai *et al.*, “Qwen3-vl technical report,” *arXiv preprint arXiv:2511.21631*, 2025.
- [5] R. Krishna *et al.*, “Visual genome: Connecting language and vision using crowdsourced dense image annotations,” *International Journal of Computer Vision*, vol. 123, pp. 32–73, 2017.

- [6] D. A. Hudson and C. D. Manning, “Gqa: A new dataset for real-world visual reasoning and compositional question answering,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 6700–6709.
- [7] A. Kamath, M. Singh, Y. LeCun, G. Synnaeve, I. Misra, and N. Carion, “MDETR: Modulated detection for end-to-end multi-modal understanding,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 1780–1790.
- [8] S. Liu *et al.*, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *Computer Vision – ECCV 2024*, 2024, pp. 38–55.
- [9] Z. Peng *et al.*, “Kosmos-2: Grounding multimodal large language models to the world,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [10] H. You *et al.*, “Ferret: Refer and ground anything anywhere at any granularity,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [11] B. Chen *et al.*, “SpatialVLM: Endowing vision-language models with spatial reasoning capabilities,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 14455–14465.
- [12] C. Sima *et al.*, “DriveLM: Driving with graph visual question answering,” in *Computer Vision – ECCV 2024*, 2024, pp. 256–274.
- [13] X. Tian *et al.*, “DriveVLM: The convergence of autonomous driving and large vision-language models,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 4698–4726.
- [14] H. Shao *et al.*, “LMDrive: Closed-loop end-to-end driving with large language models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 15120–15130.
- [15] B. Jiang *et al.*, “Senna: Bridging large vision-language models and end-to-end autonomous driving,” *International Journal of Computer Vision*, vol. 134, no. 9, 2026.
- [16] NVIDIA *et al.*, “Alpamayo-R1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail,” *arXiv preprint arXiv:2511.00088*, 2025.
- [17] Y. Li *et al.*, “ReCogDrive: A reinforced cognitive framework for end-to-end autonomous driving,” in *International Conference on Learning Representations (ICLR)*, 2026.
- [18] W. Zheng *et al.*, “DriveAgent-R1: Advancing vlm-based autonomous driving with active perception and hybrid thinking,” in *International Conference on Learning Representations (ICLR)*, 2026.
- [19] F. Codevilla, M. Muller, A. Lopez, V. Koltun, and A. Dosovitskiy, “End-to-end driving via conditional imitation learning,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 4693–4700.
- [20] D. Chen, B. Zhou, V. Koltun, and P. Krahenbuhl, “Learning by cheating,” in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100. PMLR, 2020, pp. 66–75.
- [21] P. Wu, X. Jia, L. Chen, J. Yan, H. Li, and Y. Qiao, “Trajectory-guided control prediction for end-to-end autonomous driving: A simple yet strong baseline,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [22] H. Li, Y. Zhao, J. Zhong, B. Wang, C. Sun, and F. Sun, “Delving into the secrets of BEV 3d object detection in autonomous driving: A comprehensive survey,” *IEEE Trans. Intell. Transp. Syst.*, vol. 27, no. 1, pp. 119–144, 2026.
- [23] S. Hu, L. Chen, P. Wu, H. Li, J. Yan, and D. Tao, “ST-P3: End-to-end vision-based autonomous driving via spatial-temporal feature learning,” in *Computer Vision – ECCV 2022*, 2022, pp. 533–549.
- [24] K. Renz, K. Chitta, O.-B. Mercea, A. S. Koepke, Z. Akata, and A. Geiger, “PlanT: Explainable planning transformers via object-level representations,” in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 459–470.
- [25] Y. Hu *et al.*, “Planning-oriented autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 17853–17862.
- [26] B. Jiang *et al.*, “VAD: Vectorized scene representation for efficient autonomous driving,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 8340–8350.
- [27] W. Sun, X. Lin, Y. Shi, C. Zhang, H. Wu, and S. Zheng, “SparseDrive: End-to-end autonomous driving via sparse scene representation,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 8795–8801.
- [28] B. Liao *et al.*, “DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 12037–12047.
- [29] D. Zhang *et al.*, “CarPlanner: Consistent auto-regressive trajectory planning for large-scale reinforcement learning in autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 17239–17248.
- [30] R. Feng *et al.*, “ARTEMIS: Autoregressive end-to-end trajectory planning with mixture of experts for autonomous driving,” *IEEE Robotics and Automation Letters*, vol. 11, no. 1, pp. 226–233, 2026.
- [31] Y. Li *et al.*, “DriveVLA-W0: World models amplify data scaling law in autonomous driving,” in *International Conference on Learning Representations (ICLR)*, 2026.
- [32] X. Zhou *et al.*, “Qwen-Drive-1.0: An initial step towards a vision-language foundation model for autonomous driving,” *arXiv preprint arXiv:2609.00111*, 2026.
- [33] J. Phillion, X. B. Peng, and S. Fidler, “Trajenglish: Traffic modeling as next-token prediction,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [34] Z. Yang, L. Chen, Y. Sun, and H. Li, “Visual point cloud forecasting enables scalable autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 14673–14684.
- [35] Y. Bai *et al.*, “Is a 3D-tokenized LLM the key to reliable autonomous driving?” *arXiv preprint arXiv:2405.18361*, 2024.
- [36] X. Huang *et al.*, “DriveGPT: Scaling autoregressive behavior models for driving,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 25908–25921.
- [37] K. Pertsch *et al.*, “FAST: Efficient action tokenization for vision-language-action models,” in *Robotics: Science and Systems (RSS)*, 2025.
- [38] C. Chen *et al.*, “Devil is in narrow policy: Unleashing exploration in driving VLA models,” *arXiv preprint arXiv:2603.06049*, 2026.
- [39] K. Renz, L. Chen, E. Arani, and O. Sinavski, “SimLingo: Vision-only closed-loop autonomous driving with language-action alignment,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 11993–12003.
- [40] X. Wang *et al.*, “Unifying language-action understanding and generation for autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026, pp. 25193–25203.
- [41] X. Gao *et al.*, “Neuro-symbolic drive: Rule-grounded faithful reasoning for driving VLAs,” *arXiv preprint arXiv:2606.23938*, 2026.
- [42] F. Yu *et al.*, “BDD100K: A diverse driving dataset for heterogeneous multitask learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 2636–2645.
- [43] H. Caesar *et al.*, “nuScenes: A multimodal dataset for autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11621–11631.
- [44] P. Sun *et al.*, “Scalability in perception for autonomous driving: Waymo open dataset,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 2446–2454.
- [45] H. Caesar *et al.*, “nuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles,” *arXiv preprint arXiv:2106.11810*, 2021.
- [46] T. Deruyttere, S. Vandenheide, D. Grujicic, L. Van Gool, and M.-F. Moens, “Talk2Car: Taking control of your self-driving car,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 2088–2098.
- [47] T. Qian, J. Chen, L. Zhuo, Y. Jiao, and Y.-G. Jiang, “NuScenes-QA: A multi-modal visual question answering benchmark for autonomous driving scenario,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 5, 2024, pp. 4542–4550.
- [48] A.-M. Marcu *et al.*, “LingoQA: Visual question answering for autonomous driving,” in *Computer Vision – ECCV 2024*, 2024, pp. 252–269.
- [49] S.-Y. Park *et al.*, “NuPlanQA: A large-scale dataset and benchmark for multi-view driving scene understanding in multi-modal large language models,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 8066–8076.
- [50] J.-T. Zhai *et al.*, “Rethinking the open-loop evaluation of end-to-end autonomous driving in nuScenes,” *arXiv preprint arXiv:2305.10430*, 2023.
- [51] D. Dauner *et al.*, “NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 28706–28719.

- [52] X. Jia, Z. Yang, Q. Li, Z. Zhang, and J. Yan, "Bench2Drive: Towards multi-ability benchmarking of closed-loop end-to-end autonomous driving," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [53] W. Cao *et al.*, "Pseudo-simulation for autonomous driving," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 4709–4722.
- [54] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 15. PMLR, 2011, pp. 627–635.
- [55] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-training with noisy student improves ImageNet classification," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10687–10698.
- [56] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 15 476–15 488.
- [57] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [58] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 53 728–53 741.
- [59] T. Chu *et al.*, "SFT memorizes, RL generalizes: A comparative study of foundation model post-training," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 10 818–10 838.
- [60] Z. Shao *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.
- [61] Q. Yu *et al.*, "DAPO: An open-source LLM reinforcement learning system at scale," in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 125 532–125 554.
- [62] G. Cui *et al.*, "The entropy mechanism of reinforcement learning for reasoning language models," in *International Conference on Learning Representations (ICLR)*, 2026.
- [63] B. Jiang, S. Chen, Q. Zhang, W. Liu, and X. Wang, "AlphaDrive: Unleashing the power of VLMs in autonomous driving via reinforcement learning and reasoning," *arXiv preprint arXiv:2503.07608*, 2025.
- [64] Z. Huang *et al.*, "Gen-Drive: Enhancing diffusion generative driving policies with reward modeling and reinforcement learning fine-tuning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 3445–3451.
- [65] X. Weng, B. Ivanovic, Y. Wang, Y. Wang, and M. Pavone, "PARA-Drive: Parallelized architecture for real-time autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 15 449–15 458.
- [66] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, "TransFuser: Imitation with transformer-based sensor fusion for autonomous driving," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 12 878–12 895, 2023.
- [67] Z. Sheng, X. Ye, J. Luo, S. Chen, and L. Ren, "ExploreVLA: Dense world modeling and exploration for end-to-end autonomous driving," *arXiv preprint arXiv:2604.02714*, 2026.
- [68] Y. Luo *et al.*, "AdaThinkDrive: Adaptive thinking via reinforcement learning for autonomous driving," *arXiv preprint arXiv:2509.13769*, 2025.
- [69] OpenAI, "GPT-5.4 Thinking System Card," March 2026, published March 5, 2026. [Online]. Available: <https://openai.com/index/gpt-5-4-thinking-system-card/>
- [70] Kimi Team *et al.*, "Kimi K2.5: Visual agentic intelligence," *arXiv preprint arXiv:2602.02276*, 2026.



Xiao Liu received the M.S. degree in vehicle engineering from the Harbin Institute of Technology, Harbin, China, in 2021. He is pursuing the Ph.D. degree in intelligent vehicle technology at the Beijing Institute of Technology, Beijing, China. He was a research intern at Baidu and Shineon AI, working on large vision-language models for autonomous driving. His research interests include vision-based perception, end-to-end autonomous driving, and large language models for complex driving scenarios.



Haoyu Li is currently pursuing the Ph.D. degree at the National Engineering Research Center of Electric Vehicles, Beijing Institute of Technology, China. He is also a visiting Ph.D. student at the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. His research interests include robust multi-sensor fusion for perception and prediction in autonomous driving, and embodied intelligence for physical-world agents.



Jianghao Leng received the B.S. degree in vehicle engineering from Beijing Institute of Technology, Beijing, China, in 2019, and the Ph.D. degree in mechanical engineering from Beijing Institute of Technology in 2025. From 2023 to 2024, he was a visiting student at the College of Design and Engineering, National University of Singapore. His research interests include multi-sensor fusion SLAM, motion planning and eco-driving for connected and autonomous vehicles (CAVs).



Lin Wang (Member, IEEE) is currently an Assistant Professor with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore, where he leads the Embodied Perception and InterACTion (EmPACT) Lab. His research interests include bio-inspired sensing and perception, event-based vision, sensor fusion, neuromorphic multi-modal AI, spatial intelligence, embodied robotic systems, and edge AI.



Chao Sun (Member, IEEE) received the B.S. and Ph.D. degrees in mechanical engineering from Beihang University, Beijing, China, and the Beijing Institute of Technology, Beijing, China, in 2010 and 2016, respectively. He was a postdoctoral researcher with the Energy, Controls, and Applications Laboratory, University of California, Berkeley, CA, USA. He is currently an Associate Professor at the Beijing Institute of Technology. His research interests include automated and connected vehicles and hybrid electric vehicles.

APPENDIX A IMPLEMENTATION DETAILS

All stages of the progressive GRA training procedure in Sec. III-D fine-tune Qwen3-VL-8B-Instruct with DeepSpeed ZeRO-2. Each input contains the native 1920×1080 front-camera image, ego state, motion history, and navigation command. Table S1 gives the training settings.

TABLE S1
TRAINING CONFIGURATION. BATCH SIZE IS PER GPU. GA DENOTES GRADIENT ACCUMULATION.

Stage	Epochs	LR	Batch / GA	Max length	Compute
GRA pre-training	1	1×10^{-5}	4 / 1	4,096	4×8 H20
Planner warm-up	3	1×10^{-5}	4 / 1	4,096	2×8 H20
Self-distillation	1	5×10^{-6}	4 / 1	4,096	2×8 H20 1 rollout + 3 training
Active RL	—	1×10^{-6}	4 / 1	4,096	+ 1 sim node

NAVSIM data composition: Of the 107K grounded GR-NavSim scenes reported in the main paper, 102,861 have valid official evaluation caches and form the planner and Active RL pool. Grounding-consistency filtering retains 62,111 of these cases for planner warm-up.

A. Verified Self-Distillation

Starting from the planner warm-up policy, GRAVA samples complete reasoning-to-action sequences $y_i = [r_i; a_i]$ and selects candidates using the closed-loop rewards of their decoded trajectories. As defined in Eq. (20), each retained target $y_i^+ = [r_i^+; a_i^+]$ keeps the reasoning and Executable Planner action from the same reward-selected rollout. The distilled policy initializes Active RL and serves as its frozen reference.

B. Executable Planner Decoding

Algorithm S1 defines the fixed decoder $D_p(g, \phi; s_{ego})$ that realizes each autoregressive Executable Planner action as an eight-waypoint ego-frame trajectory. Each branch reads the fields declared in Table I.

The decoder has no learned parameters. STOP expands an endpoint into a smooth profile based on the current ego speed, and CRAWL decodes eight position controls. CURVE evaluates a cubic Bézier curve from the ego origin $p_0 = (0, 0)$,

$$B(t) = \sum_{j=0}^3 \binom{3}{j} (1-t)^{3-j} t^j p_j, \quad t \in [0, 1], \quad (S1)$$

at $t \in \{1/8, \dots, 1\}$. CRUISE uses a cubic Hermite profile for monotone longitudinal progress and linear interpolation for the terminal lateral offset. The current and terminal speeds condition the longitudinal profile.

Why CRAWL emits explicit waypoints: CRAWL outputs eight points for low-speed stop-and-go motion. Its coordinate range is narrower than that of unrestricted waypoint regression, so direct prediction remains tractable. The points retain small longitudinal changes that a low-dimensional curve may smooth out. The other primitives span wider motion ranges and use compact geometric parameters.

Algorithm S1 Executable Planner decoding $\hat{\tau} = D_p(g, \phi; s_{ego})$.

```

Input : structured action
       {planner: p, gear: g, params: phi},
       ego state s_ego
Output: eight-waypoint trajectory
       tau_hat = (x_1..8, y_1..8)

1: parse p, g, phi <- action
2: assert phi in Phi_p           # validate schema
3: p0 <- (0, 0)                  # ego origin
4: switch p do
5:   case STOP:
6:     q <- stop_profile(lphi.endpointll,
7:                       speed(s_ego), 8)
8:     tau_hat <- q * phi.endpoint
9:   case CRAWL:
10:    tau_hat <- decode_points(g, phi.points)
11:   case CURVE:
12:    B(t) <- Bezier(
13:      p0, decode_controls(g, phi))
14:    tau_hat <- {B(k/8)} for k = 1..8
15:   case CRUISE:
16:    y <- monotone_Hermite(
17:      phi.progress_end, speed(s_ego),
18:      phi.v_end, 8)
19:    x <- {k * phi.lat_end / 8} for k = 1..8
20:    tau_hat <- {(x_k, y_k)} for k = 1..8
21: return tau_hat                # ego frame

```

C. Active RL Configuration

Active RL scores each complete GRA reasoning-to-action sequence with the PDMS of its decoded trajectory. We sample $K = 8$ completions at temperature 0.8 and top- $p = 0.95$, set the KL coefficient to $\beta = 0.01$, and use DAPO-style asymmetric clipping at (0.20, 0.28). Rewards are normalized within each group; overlong completions are filtered, with up to three resampling attempts.

Recoverable scenario selection: We screen every training case with one greedy completion, yielding a mean normalized PDMS of 0.9101. Cases below 0.9 receive eight sampled rollouts. We retain cases that satisfy all three reward-variation criteria:

$$\min s_i < 0.8, \quad \max s_i \geq 0.9, \quad \text{std}(s_i) \geq 0.10 \quad (S2)$$

Table S2 reports the number of cases after each filter.

TABLE S2
SELECTION OF RECOVERABLE SCENARIOS FOR THE ACTIVE RL UPDATE SET.

Selection step	Cases	Meaning
Current checkpoint evaluation	102,861	One greedy completion per case
Greedy below 0.9	26,151	Cases with potential for improvement
Best-of-8 reaches at least 0.9	14,398	Recoverable under best-of-8 sampling
All reward-variation criteria	4,676	Cases used for the RL update

Low greedy reward alone does not define the training set. We also require low- and high-reward sampled outcomes and sufficient variance. Consecutive active sets of 4,676 and 6,464 cases share only 1,995 cases, a Jaccard overlap of 21.8%, motivating rescreening after every outer loop.

APPENDIX B

VISUALIZATION OF GRA PLANNING



Fig. S1. Expanded qualitative GRA planning examples for Fig. 8 on NAVSIM: (a) signal-controlled STOP, (b) CRAWL for a queue and crossing rider, (c) CRUISE as the queue resumes, and (d) CURVE around an open roadside door.



Fig. S2. Qualitative GRA planning on the internal benchmark: (a) STOP behind a lead vehicle at a red signal, (b) CRUISE with lateral clearance from a construction zone, (c) CRAWL in a wet-road queue, and (d) CURVE for a left turn constrained by a pedestrian.

APPENDIX C ACTIVE RL INFRASTRUCTURE

Active RL requires trajectory-level rewards for complete autoregressive sequences. We therefore separate rollout generation, simulator scoring, and policy optimization into three services while keeping the reasoning and action tokens within the same rollout.

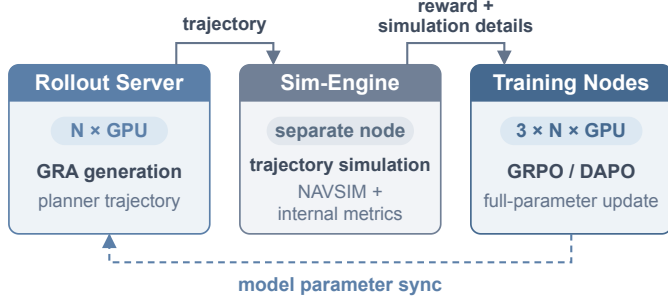


Fig. S3. Distributed infrastructure for Active RL. The rollout server sends decoded trajectories to the sim-engine, which returns rewards and simulation metadata to the training nodes. After each update, the training nodes synchronize the model with the rollout server.

The rollout server runs vLLM on one node with eight H20 GPUs, tensor parallelism, a 4,096-token limit, and prefix caching. The trainer updates all parameters on three nodes with eight H20 GPUs each using PyTorch distributed data parallelism and DeepSpeed ZeRO-2. A separate sim-engine node stores scene caches and runs the dataset-specific evaluators.

a) Execution flow: For each prompt, the rollout server samples eight reasoning-to-action sequences with the settings in Section A-C. The fixed decoder converts each valid action into an eight-waypoint trajectory. The server sends the trajectory and the scene metadata required by the evaluator to the sim-engine.

The sim-engine returns a scalar reward and simulation details keyed by the request identifier. The trainer updates a prompt group after the sim-engine scores all eight completions. An unparseable action receives zero reward. The trainer then normalizes rewards within the group and applies Eq. (27) of the main paper. It filters overlong completions and resamples zero-variance groups under the limits in Section A-C. The training nodes synchronize the model with the rollout server after each update.

b) Simulator scoring API: The sim-engine uses one request format for two scoring modes. The NAVSIM and internal modes return the aggregate and component scores defined in Sections IV-A and IV-B of the main paper, respectively. The two modes share the rollout and training code but use different scene loaders and score adapters.

The reward client caches each simulator response so all reward components reuse one evaluation. Active-set screening, online RL, and checkpoint evaluation use the same endpoints. The sim-engine runs independently of distributed training.

APPENDIX D ANNOTATION-TIME ALIASES AND TRAINING-TIME GROUNDING

GRA must preserve object identity across two interfaces: the annotation LLM and perception tools exchange compact aliases, whereas training requires image-region references. We therefore convert each alias to a bounding-box token before constructing

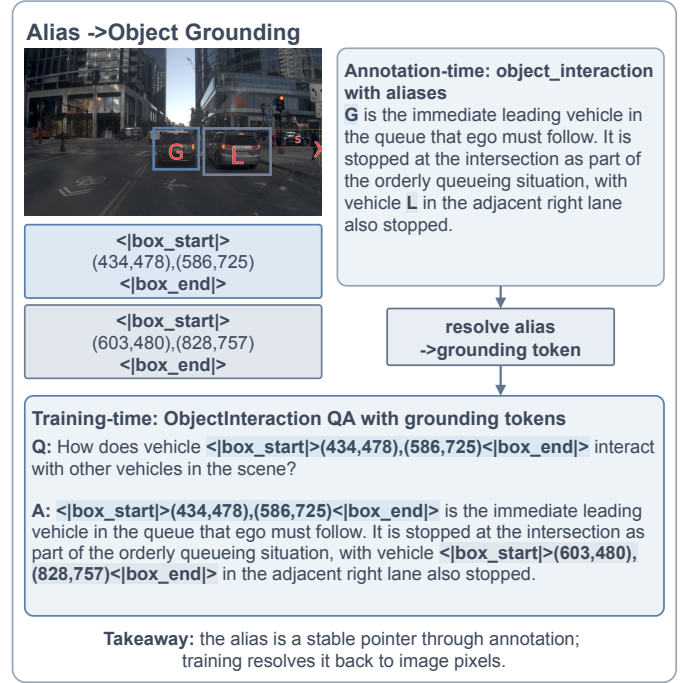


Fig. S4. Alias resolution from annotation to training. Short aliases maintain object identity across annotation steps; training replaces them with bounding-box tokens referring to the same image regions, preserving the interaction content.

training targets. Figure S4 shows this transition for objects G and L without changing their interaction content.

a) Annotation-time: Each key object receives a short alias, such as G or L, that links the outputs of the annotation LLM and perception tools across graph-construction steps. Surround-view cameras and a LiDAR-assisted perception tool provide its 3D ego-frame state. Let $\pi_c(P_j)$ denote the two-dimensional image extent obtained by projecting the 3D box P_j into camera c . We retain the object only if this extent intersects a valid camera image:

$$\mathcal{O}_{\text{ann}} = \{j : \exists c \in \mathcal{S}_{\text{ann}}, \pi_c(P_j) \cap \Omega_c \neq \emptyset \text{ and } z_c(P_j) > 0\}, \quad (\text{S3})$$

where $\mathcal{S}_{\text{ann}}$ is the surround-view camera set, Ω_c is the image extent of camera c , and $z_c(P_j) > 0$ requires the object to be in front of that camera. Objects outside every camera view are dropped before aliases are assigned.

b) Training-time: Training may use a different camera configuration from the annotation rig, such as a single front camera or a smaller surround-view set. We therefore reproject each annotated object into the training cameras $\mathcal{S}_{\text{train}}$ and keep only those still visible:

$$\mathcal{O}_{\text{train}} = \{j \in \mathcal{O}_{\text{ann}} : \exists c \in \mathcal{S}_{\text{train}}, \pi_c(P_j) \cap \Omega_c \neq \emptyset \text{ and } z_c(P_j) > 0\}. \quad (\text{S4})$$

For each retained object, we resolve its alias to a normalized bounding-box token $b_j \in [0, 1000]^4$ and discard references outside the training view. Annotation intermediates may contain aliases, but the resulting cognition and planning targets use bounding-box tokens. Only the reference form changes: the object-specific interaction and decision content remains intact. Section E shows this training-time form in complete questions, answers, and reasoning chains.

APPENDIX E

COMPLETE GRA QA EXAMPLE

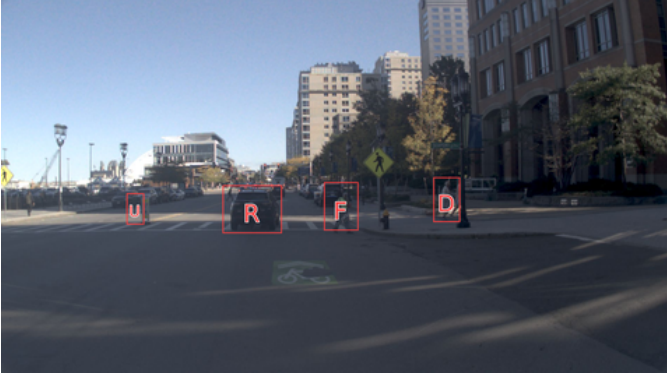


Fig. S5. Front-view observation shared by all 30 QA pairs in the complete GRA transcript. The visual labels map to normalized boxes: R: (331,502),(418,629); F: (482,494),(532,622); D: (645,481),(684,600); U: (187,523),(213,606).

The shared-supervision design in Sec. III-B derives cognition targets from the same action-relevant graph used for planning. Figure S5 and the transcript below show this projection for one NAVSIM frame through all 30 associated GRA QA pairs. The questions are grouped by graph node type and follow its dependency order: scene context, grounded objects, interactions, object-level decisions, and the ego decision. Object references in the questions, answers, and reasoning chains use normalized bounding-box tokens of the form `<box_start>(x1,y1),(x2,y2)<box_end>`, with coordinates on a 0–1000 scale. The letter overlays in Figure S5 serve only as visual lookup labels; the transcript uses the corresponding boxes directly.

Scene Context

Q Please describe the current scene environment and the ego vehicle's status.

A The ego vehicle is approaching an intersection in an urban environment with multi-lane roads. The front view shows a wide intersection with buildings on the right side and open plaza area on the left. The ego vehicle is positioned at the bottom of the image, preparing to execute a right turn. Multiple vehicles are visible in the scene: vehicle `<box_start>(331,502),(418,629)<box_end>` is directly ahead in the same lane, vehicle `<box_start>(482,494),(532,622)<box_end>` is in the forward-left position, vehicle `<box_start>(645,481),(684,600)<box_end>` is on the right side, and vehicle `<box_start>(187,523),(213,606)<box_end>` is on the far left. The road structure shows a typical urban intersection with crosswalks. The ego vehicle is currently moving at low speed (3.8 m/s $\approx$ 14 km/h) with positive acceleration, indicating it is proceeding with the right turn maneuver. The future trajectory shows a smooth rightward curve consistent with a right turn navigation intent. No significant occlusion risks are apparent, but vehicle `<box_start>(331,502),(418,629)<box_end>` ahead may require monitoring as the ego proceeds through the turn.

Q What traffic signs are visible in the current scene?

A A yellow diamond-shaped pedestrian crossing sign is visible in the front view on the right side of the road, indicating a crosswalk area ahead. No other clearly visible traffic signs, speed limit signs, or traffic lights are discernible in the provided images.

Scene Context

continued

Q What navigation maneuver is the ego vehicle executing?

A The ego vehicle is executing a TURN RIGHT maneuver at the intersection. Based on visual observation and available information, the navigation path follows a right-turn curve. Vehicle `<box_start>(331,502),(418,629)<box_end>` is positioned directly ahead on the navigation line in the same lane, approximately 15-20 meters ahead. Vehicle `<box_start>(482,494),(532,622)<box_end>` is located to the left-front of the ego vehicle, and vehicle `<box_start>(645,481),(684,600)<box_end>` is on the right side. The ego vehicle should maintain its current lane position and follow the right-turn trajectory while monitoring vehicle `<box_start>(331,502),(418,629)<box_end>` ahead.

Q Please describe the current road congestion situation ahead of the ego vehicle.

A The ego vehicle is currently BEFORE INTERSECTION (approaching the stop line/crosswalk area). This is a queueing situation rather than congestion - vehicles are orderly positioned waiting to proceed through the intersection. In the front view and visible in the image, vehicle `<box_start>(331,502),(418,629)<box_end>` is queueing directly ahead in the same lane. Vehicle `<box_start>(482,494),(532,622)<box_end>` is in the left adjacent lane, and vehicle `<box_start>(645,481),(684,600)<box_end>` is positioned to the right. There are approximately 3-4 vehicles visible in the immediate queue ahead of the ego vehicle (including `<box_start>(331,502),(418,629)<box_end>`). The left lane also contains vehicles (`<box_start>(482,494),(532,622)<box_end>` and others further ahead). The right lane appears to have vehicle `<box_start>(645,481),(684,600)<box_end>`. The low speed (3.8 m/s) and positive acceleration suggest the queue is beginning to move or the ego is approaching the intersection control point. No abnormal congestion or blockage is observed - this is normal intersection queueing behavior.

Grounded Object Perception

Q What are the location, distance, speed, and heading of vehicle `<box_start>(331,502),(418,629)<box_end>`?

A CAR type, coordinates [-3.26, 23.78, 0.44], distance 24.0m from ego, speed 6.17m/s, heading 0.05 rad (approximately straight). `<box_start>(331,502),(418,629)<box_end>` is the leading vehicle on the navigation path, occupying the ego vehicle's driving corridor directly ahead.

Q What are the location, distance, speed, and heading of pedestrian `<box_start>(482,494),(532,622)<box_end>`?

A PEDESTRIAN, coordinates [0.25, 21.85, 0.59], distance 21.9m from ego, speed 1.37m/s, heading -2.15 rad. Located near the crosswalk area forward-left of the ego vehicle, moving at walking speed.

Q What are the location, distance, speed, and heading of pedestrian `<box_start>(645,481),(684,600)<box_end>`?

A PEDESTRIAN, coordinates [4.57, 23.62, 0.78], distance 24.1m from ego, speed 1.53m/s, heading -1.85 rad. Located on the right side of the turn path near the crosswalk area, moving slowly across the intersection.

Q Which obstacles in the current scene require the most attention?

A Critical obstacles are `<box_start>(331,502),(418,629)<box_end>`, `<box_start>(482,494),(532,622)<box_end>`, and `<box_start>(645,481),(684,600)<box_end>`: `<box_start>(331,502),(418,629)<box_end>` is a CAR directly ahead occupying the navigation path during the right turn, approximately 24 meters away and moving at 6.17 m/s, acting as the leading vehicle that limits the ego's maximum speed; `<box_start>(482,494),(532,622)<box_end>` is a PEDESTRIAN located near the crosswalk forward-left of the ego vehicle at 21.9 meters, moving at walking speed and requiring caution as the ego traverses the crosswalk area; `<box_start>(645,481),(684,600)<box_end>` is a PEDESTRIAN positioned on the right side of the turn path at 24.1 meters, also requiring monitoring to ensure safe passage through the intersection.

Object Interactions

Vehicle `<box_start>(331,502),(418,629)<box_end>`.

Q What is the current intention of vehicle `<box_start>(331,502),(418,629)<box_end>`?

A Proceeding straight through the intersection at steady speed, following the traffic flow ahead.

Q Where is vehicle `<box_start>(331,502),(418,629)<box_end>` positioned relative to the ego's navigation path?

A Located in the same lane ahead of the ego vehicle, directly on the navigation path for the right turn. Based on visual analysis from the camera view, there is no lane line separation between ego and `<box_start>(331,502),(418,629)<box_end>`, confirming `<box_start>(331,502),(418,629)<box_end>` is the leading vehicle in the ego's lane.

Object Interactions continued

Q How is vehicle <box_start>(331,502),(418,629)<box_end> interacting with other nearby objects?

A Following vehicles ahead in the traffic queue, with vehicle <box_start>(482,494),(532,622)<box_end> positioned to the left-front and vehicle <box_start>(645,481),(684,600)<box_end> to the right, forming part of the multi-vehicle intersection queue.

Q What is vehicle <box_start>(331,502),(418,629)<box_end>'s position relative to the map?

A Located in the same lane ahead of the ego vehicle, directly on the navigation path for the right turn. Based on visual analysis from the camera view, there is no lane line separation between ego and <box_start>(331,502),(418,629)<box_end>, confirming <box_start>(331,502),(418,629)<box_end> is the leading vehicle in the ego's lane.

Q What impact does vehicle <box_start>(331,502),(418,629)<box_end> have on the ego vehicle?

A Acting as the leading vehicle that limits ego's maximum speed and requires maintaining safe following distance throughout the right turn maneuver.

Q What additional information is available about vehicle <box_start>(331,502),(418,629)<box_end>?

A CAR type, moving at 6.17 m/s with heading 0.05 rad (approximately straight), distance 24.0m from ego. No brake lights visible, indicating steady movement without deceleration.

Pedestrian <box_start>(482,494),(532,622)<box_end>.

Q What is the current intention of pedestrian <box_start>(482,494),(532,622)<box_end>?

A Walking across the crosswalk or moving through the pedestrian crossing area at the intersection.

Q Where is pedestrian <box_start>(482,494),(532,622)<box_end> positioned relative to the ego's navigation path?

A Located near the crosswalk that intersects with ego's planned right-turn trajectory through the intersection.

Q How is pedestrian <box_start>(482,494),(532,622)<box_end> interacting with other nearby objects?

A Moving independently in the crosswalk area without direct interaction with other vehicles.

Q What is pedestrian <box_start>(482,494),(532,622)<box_end>'s position relative to the map?

A Located in the crosswalk area visible in the image, forward-left of ego vehicle within the intersection pedestrian crossing zone, separated from ego's current lane by lane markings.

Q What impact does pedestrian <box_start>(482,494),(532,622)<box_end> have on the ego vehicle?

A Not directly on ego's planned path but located in the crosswalk area where ego must traverse during the right turn, creating a potential conflict point that requires vigilance.

Q What additional information is available about pedestrian <box_start>(482,494),(532,622)<box_end>?

A Pedestrian moving at 1.37m/s (walking speed) with heading -2.15 rad, distance 21.9m from ego, positioned near the crosswalk indicated by the pedestrian crossing sign.

Pedestrian <box_start>(645,481),(684,600)<box_end>.

Q What is the current intention of pedestrian <box_start>(645,481),(684,600)<box_end>?

A Walking across the crosswalk or along the intersection edge, moving slowly through the pedestrian crossing area.

Q Where is pedestrian <box_start>(645,481),(684,600)<box_end> positioned relative to the ego's navigation path?

A Located on the right side of ego's right-turn navigation path, within the crosswalk area near the intersection.

Q How is pedestrian <box_start>(645,481),(684,600)<box_end> interacting with other nearby objects?

A Interacting with the crosswalk environment as a pedestrian, potentially crossing the street or waiting to cross.

Q What is pedestrian <box_start>(645,481),(684,600)<box_end>'s position relative to the map?

A Located in the crosswalk/intersection area on the right side of ego's turn path, separated from ego's lane by intersection geometry, within the crosswalk zone visible in the image.

Q What impact does pedestrian <box_start>(645,481),(684,600)<box_end> have on the ego vehicle?

A Potential conflict during right turn as <box_start>(645,481),(684,600)<box_end> is positioned on the right side of the turn path; requires monitoring to ensure pedestrian does not enter the turn radius or cross in front of ego.

Object Interactions continued

Q What additional information is available about pedestrian <box_start>(645,481),(684,600)<box_end>?

A Yellow pedestrian crossing sign visible in front view indicating crosswalk presence; slow walking speed of 1.53 m/s; distance 24.1 m; heading -1.85 indicating movement across the intersection area.

Object Decisions

Q What decision did the ego vehicle make regarding vehicle <box_start>(331,502),(418,629)<box_end>, and what is the reason?

A Think: The ego vehicle is executing a right turn maneuver at the intersection. Vehicle <box_start>(331,502),(418,629)<box_end> is identified as the leading vehicle positioned directly ahead on the navigation path, approximately 24 meters away, traveling at 6.17 m/s in approximately the same direction (heading 0.05). As the critical obstacle occupying the driving corridor ahead with no lane line separation, <box_start>(331,502),(418,629)<box_end> constrains the ego vehicle's maximum allowable speed and requires the ego to maintain a safe following distance. Since <box_start>(331,502),(418,629)<box_end> is proceeding steadily through the intersection without indications of stopping, lane changing, or deviating from the path, the appropriate decision is to follow <box_start>(331,502),(418,629)<box_end>, adjusting the ego's speed to match the traffic flow while maintaining safe separation throughout the right turn completion.

Chain: scene_context.navigation (TURN RIGHT) | scene_context.summary (<box_start>(331,502),(418,629)<box_end> directly ahead on navigation line) -> <box_start>(331,502),(418,629)<box_end>.map_interaction (same lane, no separation) | <box_start>(331,502),(418,629)<box_end>.motion_state (speed 6.17m/s, heading 0.05) -> <box_start>(331,502),(418,629)<box_end>.ego_interaction (leading vehicle constraint) -> Follow

Answer: Follow

Q What decision did the ego vehicle make regarding pedestrian <box_start>(482,494),(532,622)<box_end>, and what is the reason?

A Think: The ego vehicle is executing a right turn at an intersection where a pedestrian crossing sign is visible, indicating a crosswalk ahead. Critical obstacle <box_start>(482,494),(532,622)<box_end> is a pedestrian located forward-left near this crosswalk area, moving at a walking speed of 1.37m/s. Although <box_start>(482,494),(532,622)<box_end> is not directly on the ego vehicle's planned trajectory (on_ego_path: false), the right turn maneuver requires the ego vehicle to traverse the crosswalk area where the pedestrian is present. Pedestrians in crosswalks have the right of way, and the combination of the turning maneuver and the pedestrian's proximity to the crosswalk creates a potential conflict situation. Therefore, the ego vehicle must exercise caution when proceeding with the right turn to monitor the pedestrian's movements and ensure safety.

Chain: scene_context.traffic_sign | scene_context.navigation -> <box_start>(482,494),(532,622)<box_end>.map_interaction | <box_start>(482,494),(532,622)<box_end>.motion_state -> <box_start>(482,494),(532,622)<box_end>.ego_interaction -> decision

Answer: Caution

Q What decision did the ego vehicle make regarding pedestrian <box_start>(645,481),(684,600)<box_end>, and what is the reason?

A Think: The front view shows a yellow diamond-shaped pedestrian crossing sign on the right side, indicating a designated crosswalk area ahead. The visual observation confirms that obstacle <box_start>(645,481),(684,600)<box_end> is a pedestrian located in this crosswalk area on the right side of the intersection at coordinates [4.57, 23.62], approximately 24.1 meters from ego. <box_start>(645,481),(684,600)<box_end> is moving slowly at 1.53 m/s with heading -1.85. As ego executes a right turn maneuver, <box_start>(645,481),(684,600)<box_end> is positioned on the right side of the turn path. Although <box_start>(645,481),(684,600)<box_end> is not directly on the planned trajectory (on_ego_path: false), the combination of the pedestrian crossing sign, the crosswalk location, and the slow movement of the pedestrian creates a potential interaction risk during the turn. Therefore, ego must exercise caution to monitor <box_start>(645,481),(684,600)<box_end>'s movement closely, reduce speed if necessary, and ensure the pedestrian does not enter the turn path or cross in front of the vehicle, maintaining safety throughout the right turn maneuver.

Chain: scene_context.traffic_sign -> <box_start>(645,481),(684,600)<box_end>.map_interaction | <box_start>(645,481),(684,600)<box_end>.motion_state -> <box_start>(645,481),(684,600)<box_end>.ego_interaction -> Caution

Answer: Caution

Ego Decision

Q What is the ego vehicle's overall driving strategy considering all obstacles and navigation?

A Think: The ego vehicle is executing a right turn at an intersection with vehicle <box_start>(331,502),(418,629)<box_end> as the leading obstacle directly ahead on the navigation path. The Follow decision for <box_start>(331,502),(418,629)<box_end> requires adjusting ego speed to match traffic flow; since <box_start>(331,502),(418,629)<box_end> is moving faster (6.17 m/s) than ego's current speed (3.8 m/s), this necessitates acceleration to maintain appropriate following distance. Pedestrians <box_start>(482,494),(532,622)<box_end> and <box_start>(645,481),(684,600)<box_end> are positioned near crosswalk areas on the left and right sides of the turn path respectively. While neither is directly on the ego trajectory, the Caution decisions for both indicate potential interaction risks during the right turn, requiring heightened vigilance. The trajectory intent explicitly specifies turn_right laterally and accelerate longitudinally. Synthesizing these factors: ego must execute the right turn maneuver while accelerating to follow the leading vehicle, but remain prepared to react to the pedestrians in the crosswalk zones.

Chain: <box_start>(331,502),(418,629)<box_end>.decision(Follow) | <box_start>(482,494),(532,622)<box_end>.decision(Caution) | <box_start>(645,481),(684,600)<box_end>.decision(Caution) | scene_context.navigation(TURN_RIGHT) | trajectory_intent(turn_right, accelerate) -> ego_decision(turn_right, accelerate)

Answer: lateral: turn_right | longitudinal: accelerate

APPENDIX F

GROUNDING QUESTION-ANSWERING EVALUATION PROTOCOL

A. Evaluation Set and Judge

The class-balanced held-out set tests the cognition capabilities supporting Sec. V-E and does not overlap with training data. Following Sec. D, all object references are converted to bounding boxes; samples without a valid box in the evaluation view are excluded before inference. The same normalized image region is rendered in each model's native box syntax.

MiniMax-M3 scores open-ended answers at temperature 0 and is not evaluated itself. This text-only judge receives the QA type, subtype, scoring focus, question, reference, and candidate answer. Decision subtypes instead use label accuracy.

B. Scoring Definitions

Each subtype uses the scoring rule associated with the capability it measures. All reported scores range from 0 to 10.

a) *Scene and interaction (LLM judge):* The judge scores grounding from 0 to 2, semantic correctness from 0 to 4, reasoning alignment from 0 to 2, and factual consistency from 0 to 2. Dimensions relevant to each subtype are normalized to [0, 10]. The reference is one valid answer rather than an exhaustive checklist, so omissions and correct additions are not penalized. Contradictions, hallucinations, and vague answers lower the score.

b) *Object perception (3D localization):* For perception questions with a reference 3D position, a localization score replaces the grounding dimension. It uses the Euclidean error between predicted and reference ego-frame positions,

$$\text{loc} = 2 \exp(-\|\hat{p} - p^*\|_2 / \tau), \quad \tau = 5 \text{ m}, \quad (\text{S5})$$

where $\hat{p}$ and p^* are the predicted and reference (x, y) positions. Missing coordinates receive $\text{loc} = 0$; questions without a single target position use the judge grounding score. The localization and remaining judge dimensions are normalized to [0, 10]. All models follow the ego-frame convention in Sec. F-C.

c) *Decision (label accuracy):* Object decisions use exact match over Follow, Yield, Stop, Nudge Left, Nudge Right, Overtake, and Caution. Ego decisions use the mean exact match of a lateral label (keep_lane, nudge_left, nudge_right, lane_change, turn_left, or turn_right) and a longitudinal label (accelerate, cruise, decelerate, decelerate_to_crawl, or stop). Scores are scaled to [0, 10], and negated mentions are ignored.

C. Prompts

a) *Perception coordinate prompt:* Every model receives the same system prompt, which defines the ego-frame coordinates and the localization output format:

You are a driving-scene 3D perception assistant. Given a single front-camera image and the ego state, estimate the 3D position of the queried obstacle in the EGO VEHICLE frame (units = meters):

x = lateral offset, positive to the RIGHT
y = longitudinal distance, positive AHEAD
z = height above ground
distance = $\sqrt{x^2 + y^2}$

Answer on the first line in this exact format, then one sentence:
coordinates [x, y, z], distance <D>m, speed <S>m/s, heading <H> rad

b) *Judge prompt (open-ended subtypes):* The evaluator supplies the QA type, subtype, focus, active dimensions, question, reference, and candidate answer. The common rubric below defines scoring and JSON output:

Judge an autonomous-driving QA answer. Score only the active dimensions for this subtype:

- grounding (0-2), semantic_correctness (0-4), reasoning_alignment (0-2), factual_consistency (0-2)

Scoring philosophy - precision over recall:

- The reference is ONE valid answer, not a checklist. Do NOT penalize omissions if what the candidate says is correct.
- Correct extra information is NOT penalized.
- DO penalize statements that contradict the reference/scene: wrong decision, wrong object role, wrong signal state, hallucinated objects, self-contradiction. Each wrong claim lowers the score.
- Vague/evasive answers that do not commit get low semantic_correctness, not credit for "not being wrong".

Return JSON: {"grounding": "...", "semantic_correctness": "...", "reasoning_alignment": "...", "factual_consistency": "...", "wrong_claims": "...", "rationale": "..."}.

D. Subtype-to-GRA Mapping

The thirteen subtypes follow the GRA graph from scene context through grounded objects and interactions to decisions.

a) *Scene:* Summary covers road layout, surrounding traffic, and overall context; Traffic sign covers visible signs and control states, including their absence; Navigation covers the intended maneuver, route-lane relation, and intersection context; Congestion distinguishes free flow, normal queueing, and abnormal blockage.

b) *Object perception:* Critical object description covers grounding, type, position, distance, speed, and heading. Critical object identification asks which objects constrain ego motion and what roles they play.

c) *Object interaction:* Intention evaluates a referenced object's expected behavior. Navigation interaction asks whether it affects the ego route; object interaction covers relations with other traffic participants; map interaction covers lanes, boundaries, and road structure; and ego interaction covers its effect on ego speed, path, safety margin, or right of way.

d) *Decision:* Object decision evaluates the response to one critical object; Ego decision evaluates the lateral and longitudinal strategy after combining object-level constraints.